École des Sciences de l'Information
Rabat

Rapport de Projet de Fin d'Études

Attijariwafa Bank – ESI

Optimisation du réseau d'agences bancaires

Année universitaire : 2025/2026

Juin 2026

Étudiant : Salah Eddine AHADAF
Encadrante Académique : BOUKENZE Basma
Encadrant d'entreprise : EL RHEDDANE Ahmed
Option : Ingénierie des Connaissances et Science des Données

Dédicace

Remerciements

Résumé

Ce rapport présente le travail réalisé dans le cadre d'un projet de fin d'études mené au sein d'Attijariwafa Bank, visant à construire une plateforme d'aide à la décision pour l'optimisation du réseau d'agences bancaires sur l'ensemble du territoire marocain. Le problème central est celui de l'évaluation objective des agences existantes et de la sélection de nouveaux emplacements à fort potentiel, tout en détectant les zones de redondance qui justifient une fusion ou une fermeture.

Pour répondre à ce besoin, une approche combinant plusieurs techniques a été mise en place : collecte de points d'intérêt via OpenStreetMap, Google Maps et Overture Maps Foundation, scoring géospatial normalisé basé sur l'environnement commercial et résidentiel de chaque agence, clustering K-Means pour regrouper les agences selon leur profil de localisation, similarité cosinus vectorisée pour comparer des emplacements candidats aux agences les plus performantes, et visualisation interactive via Streamlit, Folium et Plotly.

Les résultats obtenus montrent que la plateforme est capable d'évaluer plusieurs millions de points candidats sur la carte du Maroc, de produire des recommandations géolocalisées classées par score combiné, et de mettre à disposition un tableau de bord complet pour les décideurs. Des pistes d'amélioration sont proposées en fin de rapport, notamment l'intégration de données socio-économiques supplémentaires et le renforcement des modèles par apprentissage continu.

Table des matières

Liste des figures

Table des figures

Liste des tableaux

Liste des tableaux

Liste des acronymes

Acronyme	Définition
API	Application Programming Interface
AWB	Attijariwafa Bank
BI	Business Intelligence
CSV	Comma-Separated Values
DBSCAN	Density-Based Spatial Clustering of Applications with Noise
ESI	École des Sciences de l'Information
ETL	Extract, Transform, Load
GIS	Geographic Information System
GPS	Global Positioning System
KMeans	K-Means Clustering
KPI	Key Performance Indicator
ML	Machine Learning
OSM	OpenStreetMap
OMF	Overture Maps Foundation
PCA	Principal Component Analysis
PFE	Projet de Fin d'Études
POI	Point of Interest (Point d'Intérêt)
SQL	Structured Query Language
UI	User Interface
UX	User Experience
UML	Unified Modeling Language)

Introduction Générale

Dans le contexte actuel de transformation numérique, les données occupent une place centrale dans le fonctionnement et la compétitivité des institutions financières. Que ce soit dans le secteur bancaire, la gestion patrimoniale ou les services financiers, la capacité à collecter, traiter et analyser des volumes importants d'informations géographiques et commerciales est devenue un enjeu stratégique de premier plan. Cette évolution, portée par la multiplication des sources numériques comme les points d'intérêt cartographiques, les transactions bancaires ou les indicateurs de performance, oblige les organisations financières à repenser leurs outils d'aide à la décision.

Face à cette réalité, les méthodes traditionnelles de sélection d'emplacement bancaire, basées sur des critères subjectifs et des processus souvent manuels, montrent aujourd'hui clairement leurs limites. Elles peinent à suivre la croissance rapide des réseaux, à tenir compte de la diversité des environnements urbains et ruraux, et à produire des analyses rapides et fiables. De nouvelles approches, combinant l'apprentissage automatique et la géolocalisation, émergent pour combler ce manque. Elles offrent une réponse plus adaptée aux exigences actuelles de l'expansion bancaire.

C'est dans ce contexte que s'inscrit ce stage de fin d'études, réalisé au sein d'Attijariwafa Bank. Le projet consiste à concevoir et mettre en place une plateforme d'intelligence territoriale dédiée à l'analyse du réseau d'agences bancaires. L'objectif est de fournir un outil capable d'évaluer et de comparer les agences existantes sur l'ensemble du territoire marocain, d'identifier les zones de redondance et les candidats à la fermeture ou à la fusion, et de recommander les meilleurs emplacements pour de nouvelles ouvertures. Pour cela, la plateforme s'appuie sur plusieurs scores normalisés construits à partir de l'environnement immédiat de chaque agence : score commercial, score bureaux, score résidentiel, accessibilité, concurrence et cannibalisation.

La solution technique repose sur une architecture modulaire, intégrant la collecte de points d'intérêt depuis plusieurs sources cartographiques, des algorithmes de clustering et de similarité vectorielle, et un tableau de bord interactif développé avec les outils Python les plus adaptés. Cette approche vise à produire des résultats précis, reproductibles et directement exploitables par les responsables métier.

Le rapport est organisé en plusieurs chapitres. Le premier présente le contexte du projet, la banque, la problématique et les objectifs fixés. Le deuxième décrit la conception fonctionnelle et les choix d'architecture de données. Le troisième détaille la conception technique, les algorithmes retenus et les outils utilisés. Le quatrième chapitre présente la réalisation concrète avec les résultats obtenus, les limites identifiées et les perspectives d'amélioration.

Chapitre 1

Contexte du Projet

1.1 Présentation de l'Organisme d'Accueil

Ce premier chapitre a pour but de présenter le cadre général dans lequel ce stage de fin d'études a été effectué. On va commencer par présenter Attijariwafa Bank, l'institution qui nous a accueillis, en donnant ses informations essentielles, ses chiffres clés et son histoire. Ensuite, on va présenter le projet lui-même : son contexte, le problème auquel il répond, les objectifs qu'on s'est fixés et la façon dont on a organisé le travail.

Cette présentation permet de bien comprendre pourquoi ce projet a été lancé, dans quel environnement il a été réalisé, et quelles contraintes il devait respecter. Elle prépare aussi à lire les chapitres suivants en donnant les repères nécessaires.

1.2 Présentation d'Attijariwafa Bank

Attijariwafa Bank est le premier groupe bancaire et financier au Maroc et l'un des plus importants du continent africain. Il est présent dans 26 pays à travers l'Afrique, l'Europe et le Moyen-Orient, et accompagne aujourd'hui plus de 12 millions de clients, qu'il s'agisse de particuliers, de professionnels, d'entreprises ou d'institutions.

Le groupe propose une gamme complète de services financiers : banque de détail, banque d'entreprise, financement, assurance, gestion d'actifs et services spécialisés. Cette diversité en fait un acteur incontournable dans le paysage économique marocain et africain.

Avec un réseau de plus de 4 300 agences à l'échelle du groupe, dont environ 1 196 au Maroc, et plus de 20 000 collaborateurs dans le monde, Attijariwafa Bank représente une organisation dont la gestion du réseau physique est un enjeu quotidien majeur. C'est dans cette logique que le projet prend tout son sens.



FIGURE 1.1 – AWB

1.2.1 La Fiche Signalétique

La fiche signalétique regroupe les informations administratives et générales sur l'organisme d'accueil.

Désignation	Valeur
Nom commercial	Attijariwafa Bank
Forme juridique	Société Anonyme à Conseil d'Administration
Siège social	2, Boulevard Moulay Youssef, Casablanca 20000, Maroc
Site de stage	Rue Yaacoub El Mansour, Rabat, Maroc
Site web	`www.attijariwafabank.com`
Capital social	2 151 408 390 DH
Registre de commerce	N° 333 (Casablanca)
Président Directeur Général	Mohamed El Kettani
Secteur d'activité	Services bancaires et financiers
Présence internationale	26 pays (Afrique, Europe, Moyen-Orient)

TABLE 1.1 – Fiche signalétique d'Attijariwafa Bank

1.2.2 Historique d'Attijariwafa Bank

Attijariwafa Bank est née en 2004 de la fusion de deux banques marocaines historiques : la Banque Commerciale du Maroc (BCM) et Wafabank. Cette fusion a créé le premier groupe bancaire privé du pays, avec un réseau déjà important et une base de clients solide.

Les origines des deux entités fondatrices remontent bien plus loin. Wafabank hérite d'une histoire qui commence en 1904, lorsque la Compagnie Française de Crédit et de Banque a ouvert une agence à Tanger. La BCM, de son côté, trouve ses racines en 1911 avec l'installation d'une branche de la Banque Transatlantique, également à Tanger.

Depuis 2004, le groupe a connu une expansion progressive et régulière. Il a renforcé sa présence au Maroc, développé ses filiales africaines et intégré plusieurs acquisitions dans différents pays. En 2005, il entre en Tunisie avec l'acquisition d'une participation dans la Banque du Sud, rebaptisée Attijari Bank. En 2008 et 2009, plusieurs autres banques africaines rejoignent le groupe au Sénégal, en Côte d'Ivoire, au Congo, au Gabon et au Mali.

Mohamed El Kettani dirige le groupe depuis 2007 et a conduit plusieurs plans stratégiques successifs. Le plan actuel, "@MBITIONS 2025", lancé en 2021, vise à renforcer le leadership continental de la banque, à accélérer la transformation digitale et à améliorer l'efficacité opérationnelle.

Période	Événement clé
1904	Création à Tanger de la Compagnie Française de Crédit et de Banque (ancêtre de Wafabank)
1911	Implantation de la Banque Transatlantique à Tanger (ancêtre de la BCM)
1985	Changement de nom, naissance de Wafabank
2004	Fusion BCM et Wafabank : naissance d'Attijariwafa Bank
2005	Acquisition en Tunisie, début de l'expansion africaine
2007	Mohamed El Kettani prend la direction du groupe
2008–2010	Acquisitions en Afrique subsaharienne (Sénégal, Côte d'Ivoire, Congo, Gabon, Mali)
2021	Lancement du plan stratégique *@MBITIONS 2025*

TABLE 1.2 – Grandes étapes de l'histoire d'Attijariwafa Bank

1.2.3 Les Chiffres Clés

Attijariwafa Bank affiche des indicateurs qui témoignent de son importance dans le secteur financier africain. Ces chiffres montrent à quel point la gestion du réseau physique est une question stratégique pour le groupe.

Indicateur	Valeur	Portée
Nombre d'agences au Maroc	Environ 1 196	National
Réseau total du groupe	Plus de 4 300 agences	Groupe
Nombre de clients	Plus de 12 millions	Groupe
Collaborateurs	Plus de 20 000	Groupe
Pays de présence	26 pays	International
Bénéfice net 2023	7,5 milliards de DH	Groupe
Produit net bancaire 2023	29,9 milliards de DH	Groupe
Capital social	2,15 milliards de DH	Groupe

TABLE 1.3 – Chiffres clés d'Attijariwafa Bank (données 2023–2024)

Ces données montrent clairement qu'un réseau de cette taille nécessite des outils d'analyse performants pour accompagner les décisions liées à l'implantation, à la consolidation ou à la fermeture d'agences.

1.2.4 Pôle Transformation, Innovation, Technologies et Opérations

Le pôle Transformation, Innovation, Technologies et Opérations (TITO) est l'une des directions centrales du groupe. Il est chargé de piloter la modernisation des systèmes d'information, de gouverner la donnée et de déployer les projets technologiques au service des métiers.

Ce pôle joue un rôle clé dans la transformation numérique de la banque. Il est notamment impliqué dans les projets liés à la qualité des données, à l'automatisation des processus et au développement des outils d'aide à la décision. C'est dans ce cadre que le projet de plateforme

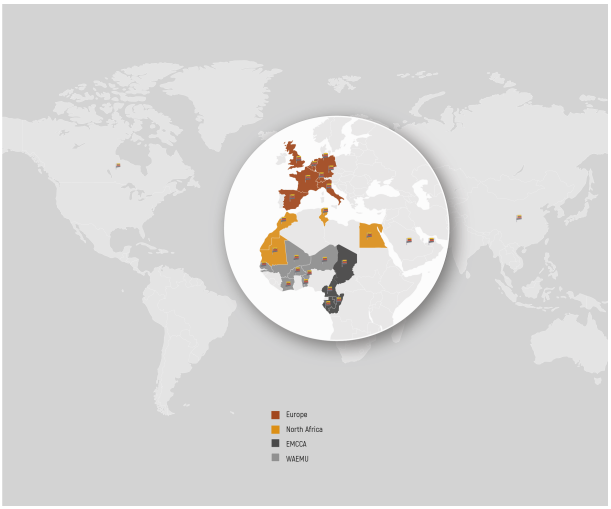


FIGURE 1.2 – FILIALES MONDIALES

d'intelligence territoriale prend sa place : il s'inscrit directement dans la logique de valorisation de la donnée pour appuyer les décisions stratégiques du réseau.

Mon stage s'est déroulé au sein du site de Yaacoub El Mansour à Rabat, en lien avec des équipes transverses concernées par la distribution bancaire et l'analyse du réseau.

1.2.5 Digital Center

Le Digital Center est l'environnement technique mis en place par Attijariwafa Bank pour développer, tester et déployer ses solutions numériques. Il constitue un cadre structuré qui garantit la sécurité, la traçabilité et la fiabilité des traitements réalisés.

Dans un contexte bancaire, ces exigences sont non négociables. Travailler dans cet environnement implique de respecter des règles précises de gestion des données, d'accès aux ressources et de validation des résultats avant toute mise en production. C'est dans ce cadre que les pipelines de traitement du projet ont été conçus et testés.

Digital Center est rattaché hiérarchiquement au Pôle Transformation, Innovation, Technologie set Opérations, l'organigramme suivant décrit les différentes relations régissant les différents pôles étantités.

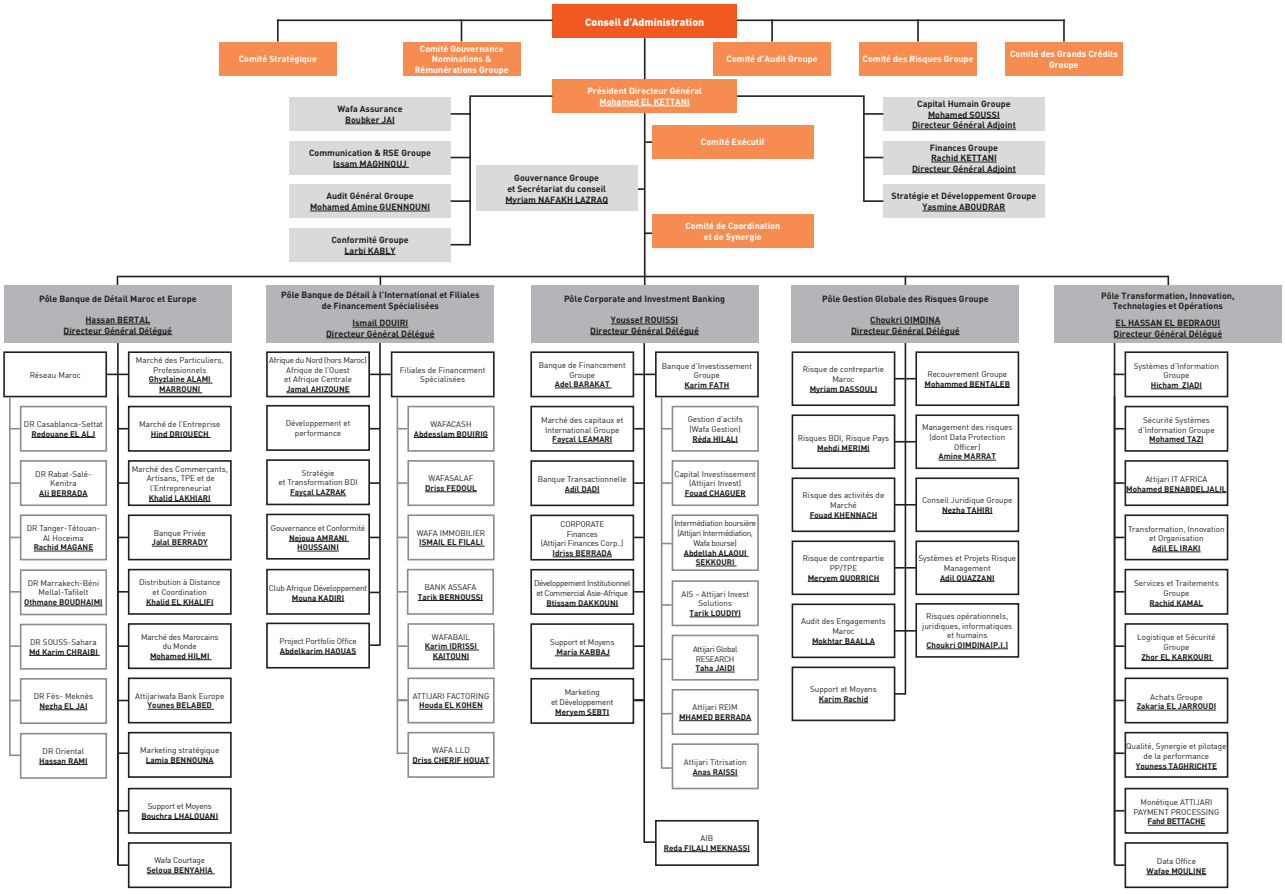


FIGURE 1.3 – Organigramme AWB

1.2.6 Cadrage du Projet

Le projet a été initié dans un contexte où la banque souhaitait mieux exploiter ses données pour appuyer les décisions relatives au réseau d'agences. L'idée de départ était de proposer un outil lisible, concret et directement utile aux équipes métier, sans nécessiter d'expertise technique particulière pour son utilisation.

Le cadrage initial a permis de définir les attentes principales, d'identifier les données disponibles et de fixer les contraintes de réalisation. Il a aussi confirmé que le projet devait rester un outil d'aide à la décision : l'objectif n'est pas d'automatiser les choix stratégiques, mais de fournir une base d'analyse solide et compréhensible.

Les grandes orientations retenues dès le départ étaient les suivantes :

— analyser le réseau existant et identifier les agences performantes ou à risque ;

— détecter les zones de chevauchement entre agences proches ;

— repérer les zones géographiques sous-exploitées avec un potentiel de développement ;

— proposer une restitution visuelle et interactive des résultats ;

— rendre le tableau de bord accessible aux décideurs sans formation technique.

1.3 Contexte du Projet

Le secteur bancaire marocain traverse une période de transformation importante. La digitalisation des services, l'évolution des habitudes des clients et l'intensification de la concurrence poussent les banques à repenser leur présence physique. Les opérations courantes migrent vers les canaux numériques, mais les agences restent indispensables pour le conseil, la gestion de projets complexes et le maintien de la relation client.

Dans ce contexte, la question de la pertinence du réseau physique devient stratégique. Certaines agences sont très bien positionnées et génèrent des revenus élevés. D'autres se retrouvent trop proches de collègues ou mal positionnées par rapport au potentiel réel du quartier ou de la ville où elles sont implantées.

Ce projet propose une réponse concrète à ce besoin en construisant une plateforme capable de transformer des données brutes et dispersées en une information structurée, visuelle et utilisable pour la prise de décision.

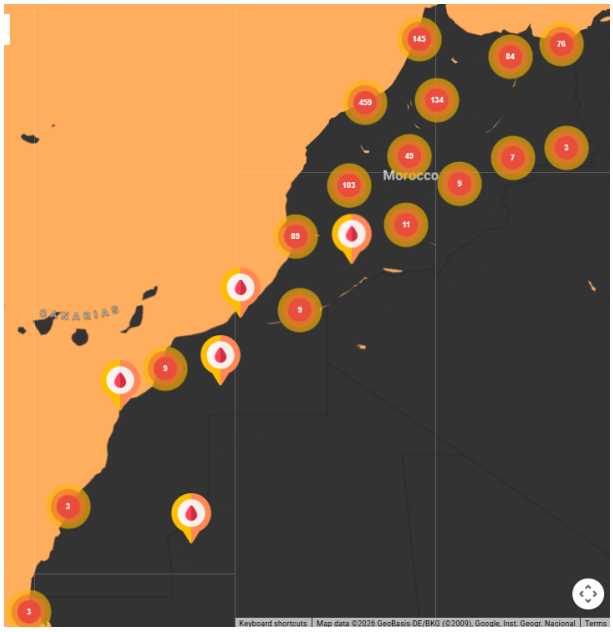


FIGURE 1.4 – Reseaux AWB

1.4 Problématique du Projet

La problématique principale peut être formulée de la façon suivante :

> *Comment aider Attijariwafa Bank à analyser et optimiser son réseau d'agences à partir de la donnée géospatiale et des points d'intérêt, afin d'améliorer la prise de décision stratégique tout en maintenant une couverture territoriale cohérente sur l'ensemble du Maroc ?*

Cette question centrale se décline en plusieurs points concrets :

— comment identifier les agences proches qui se font concurrence ou se cannibalisent mutuellement ;

— comment repérer les zones géographiques peu couvertes mais à fort potentiel commercial ;

— comment comparer objectivement des agences selon leur environnement immédiat ;

— comment présenter les résultats de manière simple, visuelle et exploitable sans expertise technique ;

— comment soutenir les décisions d'ouverture, de fermeture ou de fusion d'agences sur la base de données fiables.

Le projet n'a pas vocation à remplacer le jugement humain. Il fournit un support objectif et structuré qui permet aux responsables de prendre des décisions mieux informées et mieux documentées.

1.5 Objectifs du Projet

L'objectif général est de concevoir une solution d'aide à la décision qui permette d'analyser le réseau d'agences de façon plus précise et plus lisible qu'avec les méthodes habituelles.

Les objectifs spécifiques retenus sont les suivants :

1. collecter et centraliser les données utiles à l'analyse : coordonnées des agences, points d'intérêt environnants, données de performance synthétiques ;

2. calculer des scores normalisés qui traduisent la qualité de l'environnement de chaque agence ou de chaque emplacement candidat ;
3. regrouper les agences selon leur profil de localisation à l'aide d'algorithmes de clustering ;
4. identifier les agences proches susceptibles d'être fusionnées ou fermées ;
5. sélectionner les meilleurs emplacements potentiels pour de nouvelles ouvertures ;
6. proposer un tableau de bord interactif consultable par les équipes métier sans expertise technique.

Ces objectifs visent à faire passer l'analyse du réseau d'une observation statique à une compréhension dynamique et actionnable.

1.6 Conduite du Projet

Le projet a été conduit selon une approche progressive et itérative, ce qui est particulièrement adapté aux projets liés à la donnée. Cette façon de travailler permet de valider les résultats étape par étape et d'ajuster la direction en fonction des retours obtenus.

Les grandes phases du déroulement ont été les suivantes :

1. prise de connaissance du besoin métier et définition du périmètre ;
2. exploration et analyse des données disponibles ;
3. préparation et nettoyage des données ;
4. développement du pipeline de scoring et de clustering ;
5. développement de la sélection de nouveaux sites ;
6. construction et validation du tableau de bord interactif.

Chaque phase a fait l'objet d'une validation intermédiaire avant de passer à la suivante, ce qui a permis de garder une bonne maîtrise de l'avancement et de limiter les risques de dérives.

1.7 Méthodologie de Travail

Dans le cadre de mon projet de fin d'études portant sur l'optimisation du réseau d'agences d'Attijariwafa Bank par une approche géospatiale et machine learning, j'ai adopté une méthodologie de travail qui combine les principes de la méthode Agile avec les exigences propres aux projets de data science.

1.7.1 Choix de la méthode Agile

L'adoption de la méthode Agile s'est imposée assez naturellement pour ce type de projet, et ce pour plusieurs raisons :

— **Nature évolutive du projet :** Un projet d'analyse de données n'est pas figé dès le départ. Les résultats d'une étape orientent souvent ce qu'on fait dans la suivante, ce qui demande une méthode qui accepte les changements de direction.

— **Validation régulière :** Il était important de montrer des résultats concrets à l'encadrante et aux équipes concernées à intervalles réguliers, et non pas seulement à la toute fin du projet.

— **Gestion de l'incertitude :** Certaines étapes, comme la collecte des données ou la qualité des sources, n'étaient pas prévisibles à l'avance. La méthode Agile permet d'absorber ce genre d'imprévus sans remettre en cause tout le planning.

— **Avancement mesurable :** Travailler par petites étapes avec des livrables définis à chaque fois facilite le suivi et donne une vision claire de là où on en est.

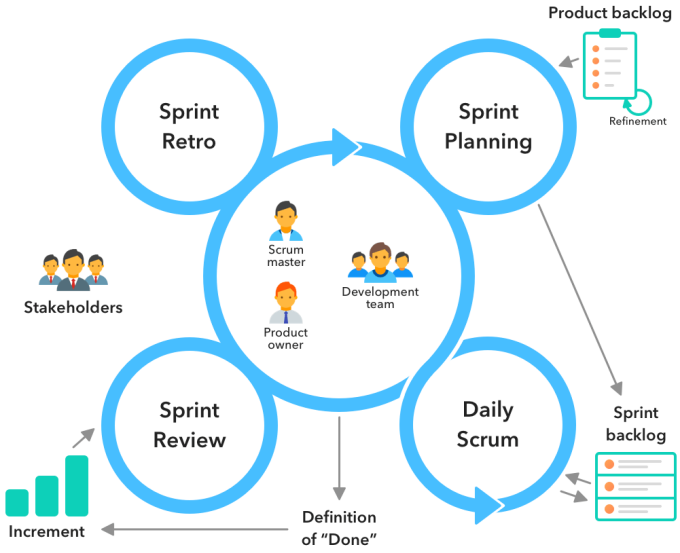


FIGURE 1.5 – Processus SCRUM et ses composants principaux

1.7.2 Organisation des sprints

Le projet a été découpé en sprints de deux semaines. Chacun avait un objectif précis et un livrable attendu, ce qui permettait de valider le travail fait avant de passer à la suite :

- **Planification du sprint :** En début de chaque sprint, on définissait les tâches à réaliser et les critères pour considérer que c'est terminé.
- **Points d'avancement réguliers :** Des échanges courts mais réguliers avec l'encadrante permettaient de signaler les blocages et de réajuster si nécessaire.
- **Revue de sprint :** À la fin de chaque sprint, les résultats étaient présentés et validés avant de lancer le suivant.
- **Rétrospective :** Un regard en arrière sur ce qui a bien fonctionné et ce qu'il fallait améliorer pour la prochaine itération.

1.7.3 Adaptation aux spécificités du Data Science

La méthode Agile a été ajustée pour tenir compte des particularités d'un projet d'analyse de données :

- **Exploration avant développement :** Avant chaque sprint de production, une phase d'exploration rapide des données était faite pour s'assurer qu'on partait sur de bonnes bases.
- **Validation visuelle des résultats :** Chaque résultat intermédiaire était vérifié visuellement, par exemple sur une carte ou un graphique, pour s'assurer qu'il avait du sens avant d'aller plus loin.
- **Documentation au fil de l'eau :** Les choix techniques et les résultats étaient notés au fur et à mesure pour faciliter la rédaction du rapport et éviter de perdre des informations importantes.
- **Plans de secours :** Pour les étapes incertaines, notamment la collecte des données via des sources externes, des alternatives avaient été identifiées en amont.

1.7.4 Outils et frameworks utilisés

Pour mener à bien ce projet, plusieurs outils ont été utilisés selon les besoins de chaque étape :

— **Python :** Langage principal utilisé pour tous les scripts de traitement, d'analyse et de modélisation.

— **Pandas et NumPy :** Pour la manipulation et le nettoyage des données.

— **Scikit-learn et SciPy :** Pour les algorithmes de clustering, de réduction de dimension et de calcul de similarité.

— **OpenStreetMap, Overture Maps et Google Maps :** Sources utilisées pour la collecte des points d'intérêt autour des agences.

— **Streamlit, Folium et Plotly :** Pour la construction du tableau de bord interactif et la visualisation des résultats sur carte.

— **Git :** Pour le suivi des versions du code tout au long du projet.

1.7.5 Rôle et responsabilités

En tant qu'étudiant stagiaire en charge du projet, j'ai assuré la totalité du développement technique, depuis la collecte initiale des données jusqu'à la livraison du tableau de bord final. Cela couvrait la conception des pipelines de traitement, la mise en place des modèles d'analyse, les tests de validation et la rédaction du présent rapport.

Les échanges réguliers avec mon encadrante académique, et avec les équipes concernées au sein de la banque, ont permis de vérifier que les choix faits correspondaient bien aux besoins réels du projet et aux attentes des parties prenantes.

1.7.6 Bénéfices observés

Cette façon de travailler s'est avérée bien adaptée au projet. Parmi les points positifs qu'on peut noter :

— **Détection rapide des problèmes :** Les anomalies dans les données ou les résultats inattendus ont été repérés tôt, ce qui a évité de les traîner jusqu'à la fin du projet.

— **Meilleure lisibilité de l'avancement :** Le découpage en sprints avec des livrables précis a rendu l'avancement du projet visible à tout moment.

— **Résultats concrets à chaque étape :** Plutôt que d'attendre la fin pour montrer quelque chose, chaque sprint produisait un résultat tangible qu'on pouvait présenter et discuter.

— **Apprentissage progressif :** Cette méthode a aussi permis de monter en compétence au fil du projet, en intégrant de nouvelles connaissances à chaque itération plutôt que d'essayer de tout apprendre d'un coup.

Cette approche s'est révélée particulièrement utile dans un environnement bancaire où les données sont sensibles et où les résultats doivent être fiables et justifiables avant d'être présentés aux décideurs.

1.8 Planification du Projet

La planification du projet a été formalisée sous forme d'un diagramme de Gantt, qui permet de visualiser l'enchaînement des tâches, leur durée et les éventuelles dépendances entre elles. Ce diagramme a servi de fil conducteur tout au long du stage pour s'assurer que le travail avançait dans les délais impartis.

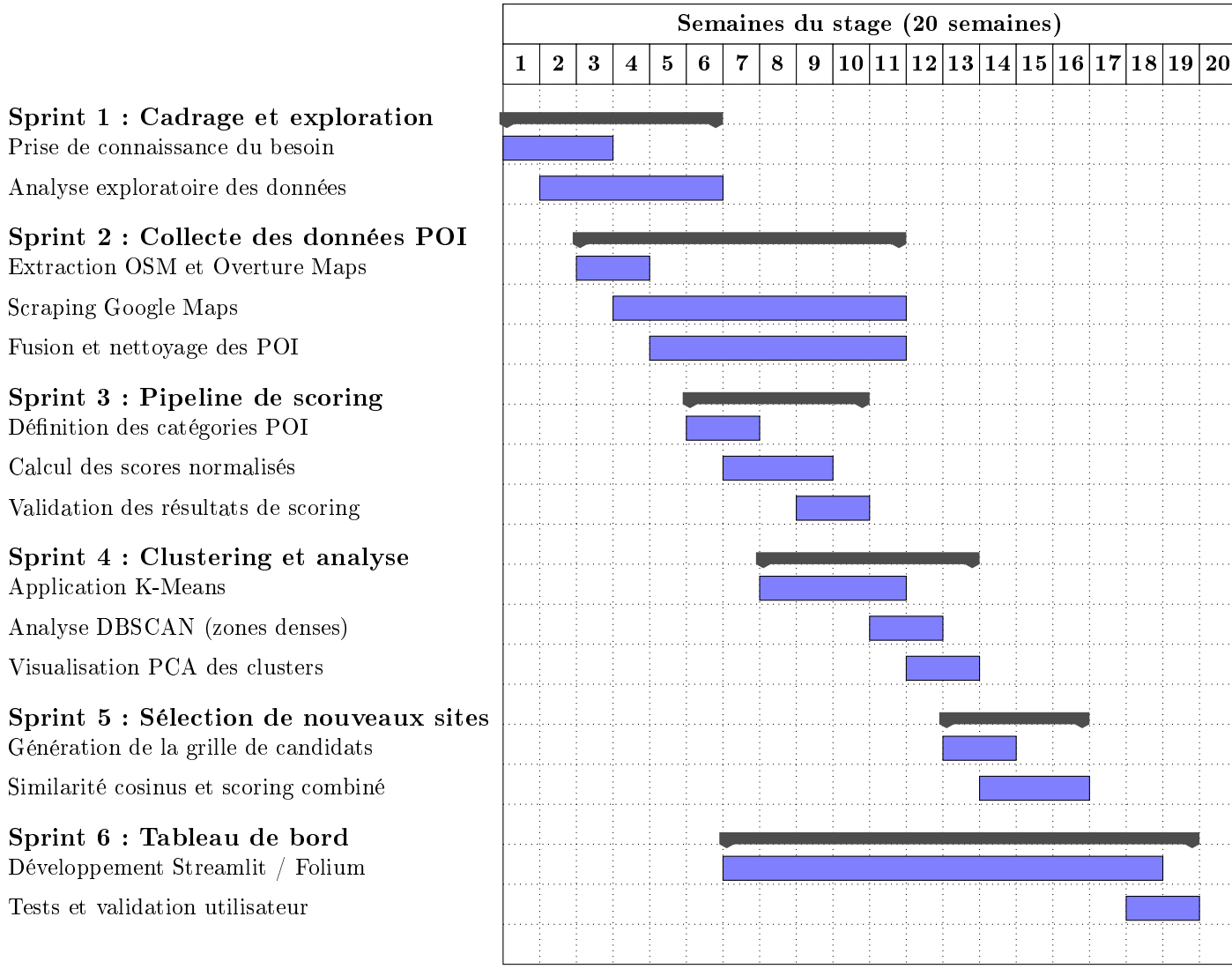


FIGURE 1.6 – Diagramme de Gantt

Le planning a été globalement respecté, avec quelques ajustements mineurs liés à la complexité de la collecte des données et au temps nécessaire pour valider la qualité des résultats intermédiaires. Le diagramme montre l'enchaînement logique des sprints et les deux jalons principaux : la livraison du pipeline de scoring à la dixième semaine et la livraison du tableau de bord à la dix-neuvième semaine.

Chapitre 2

Conception Fonctionnelle

Ce chapitre présente la conception fonctionnelle de la plateforme d'intelligence territoriale développée pour Attijariwafa Bank. Il décrit de manière détaillée le cahier des charges, les exigences fonctionnelles et non fonctionnelles, ainsi que la modélisation UML complète du système. Cette modélisation a été essentielle pour structurer le développement et garantir que la solution réponde aux besoins métier identifiés.

2.1 Présentation du Cahier des Charges

Le cahier des charges a été élaboré en collaboration avec les équipes métier d'Attijariwafa Bank et l'encadrante académique. Il définit les objectifs, le périmètre, les contraintes et les livrables attendus du projet.

2.1.1 Contexte et Justification

Attijariwafa Bank dispose d'un réseau de 1 196 agences réparties sur l'ensemble du territoire marocain. La gestion optimale de ce réseau constitue un enjeu stratégique majeur pour la banque, nécessitant des outils d'analyse sophistiqués capables d'évaluer la performance de chaque agence sur la base de son environnement géographique. Les méthodes traditionnelles de sélection d'emplacements, historiquement basées sur des critères subjectifs et des analyses manuelles, ne permettent plus de répondre à ces besoins avec la rigueur et la rapidité requises dans un contexte concurrentiel en constante évolution.

C'est dans cette optique que la plateforme a été conçue pour répondre à quatre besoins fondamentaux :

- **Évaluer** la performance de chaque agence sur la base de son environnement géographique ;
- **Identifier** les zones de redondance où plusieurs agences se cannibalisent mutuellement, réduisant ainsi leur rentabilité respective ;
- **Détecter** les zones sous-exploitées présentant un potentiel de développement important ;
- **Proposer** des emplacements optimaux pour de nouvelles ouvertures basés sur des données objectives.

2.1.2 Objectifs du Projet

L'objectif principal de ce projet est de concevoir une plateforme d'aide à la décision qui combine plusieurs composantes essentielles pour offrir une vision complète du réseau d'agences et des opportunités d'expansion. La plateforme s'articule autour de cinq axes majeurs :

1. **Collecte de données géospatiales :** Agréger des points d'intérêt (POI) provenant de sources multiples et hétérogènes telles qu'OpenStreetMap, Google Maps et Overture Maps Foundation, afin de disposer d'une base de données riche et diversifiée ;
2. **Scoring géospatial :** Calculer des scores normalisés reflétant la qualité de l'environnement de chaque agence selon sept dimensions : commercial, bureaux, résidentiel, accessibilité, concurrence, cannibalisation et densité de population (métrique par ville) ;

3. **Clustering intelligent :** Regrouper les agences selon leur profil de localisation à l'aide de l'algorithme K-Means, permettant ainsi d'identifier des patterns communs et des catégories de performance ;
4. **Sélection de sites :** Identifier les meilleurs emplacements potentiels pour de nouvelles agences via une approche innovante combinant un modèle de régression Random Forest pour la prédiction de revenu, la similarité cosinus par rapport aux meilleures agences existantes, le scoring POI et la densité de population ;
5. **Visualisation interactive :** Offrir un tableau de bord Streamlit intégrant des cartes Folium et des graphiques Plotly pour une exploration intuitive et une prise de décision éclairée.

2.1.3 Périmètre du Système

Le périmètre fonctionnel du système s'articule autour de trois axes principaux couvrant l'ensemble du flux de traitement des données.

Concernant les **entrées**, le système accepte des fichiers CSV contenant les coordonnées géographiques des agences existantes, les données de performance financière associées, les données de densité de population par ville, ainsi que les points d'intérêt mergés provenant de différentes sources de données géospatiales.

Au niveau des **traitements**, le système effectue une chaîne d'opérations complexes incluant le nettoyage et la déduplication des données, le calcul des scores multidimensionnels pour chaque agence, le clustering des agences existantes en groupes homogènes, la génération d'une grille de candidats potentiels couvrant le territoire national, l'entraînement dynamique d'un modèle Random Forest pour la prédiction de revenu, le calcul de similarité cosinus par rapport aux agences de référence, et enfin la sélection des meilleurs emplacements avec application de contraintes géographiques strictes.

Pour les **sorties**, le système produit plusieurs livrables concrets : des agences scorées accompagnées de recommandations catégorisées (KEEP, REVIEW, CLOSE), un top 20 des emplacements recommandés pour de nouvelles ouvertures, des cartes interactives permettant l'exploration visuelle des données, et des fichiers CSV téléchargeables pour analyse ultérieure ou intégration dans d'autres systèmes.

2.1.4 Contraintes

Plusieurs contraintes ont guidé la conception technique et fonctionnelle du système, imposant des choix structurants à chaque étape du développement.

Sur le plan **technique**, le développement devait respecter l'environnement Digital Center d'Attijariwafa Bank et s'appuyer exclusivement sur Python et des bibliothèques open-source éprouvées, garantissant ainsi la reproductibilité et l'indépendance vis-à-vis de solutions propriétaires coûteuses.

Les contraintes de **performance** imposaient le traitement de millions de points candidats en un temps raisonnable, spécifiquement moins de cinq minutes pour l'ensemble du pipeline de sélection, ce qui a nécessité l'utilisation de structures de données optimisées comme les arbres k-d pour les requêtes spatiales.

Les contraintes d'**utilisabilité** exigeaient une interface accessible ne nécessitant aucune expertise technique de la part des utilisateurs finaux, avec des paramètres configurables accompagnés de descriptions claires et de valeurs par défaut pertinentes.

Enfin, les contraintes liées à la **qualité des données** requéraient une gestion robuste des données manquantes par imputation ou valeurs par défaut, une déduplication efficace des POI provenant de sources hétérogènes, et une validation géographique rigoureuse assurant que tous les points candidats se situent effectivement dans les frontières du Maroc.

2.1.5 Livrables Attendus

Le projet a donné naissance à plusieurs livrables concrets, chacun couvrant une partie spécifique du pipeline de traitement.

Le **pipeline de scoring** des agences existantes, implémenté dans le fichier `model/scoring.py`, calcule les sept scores multidimensionnels et les recommandations associées pour les 1 195 agences du réseau. Le **pipeline de sélection** de nouveaux sites, contenu dans `model2/site_selection2.py`, identifie les vingt emplacements optimaux pour de futures ouvertures en combinant un modèle de régression Random Forest entraîné dynamiquement, la similarité cosinus et le scoring POI. Un **notebook de benchmarking** (`model_benchmarking.ipynb`) documente la comparaison rigoureuse entre quatre algorithmes de régression (Ridge, Random Forest, Gradient Boosting, XGBoost), démontrant la supériorité du Random Forest avec un R^2 de 0.88 en validation croisée à 5 plis. Le **tableau de bord interactif**, développé dans `model2/dashboard.py`, offre neuf onglets couvrant l'ensemble des fonctionnalités de la plateforme, de la visualisation géographique à l'exécution interactive du modèle. Les **fichiers de résultats** incluent `branches_scored.csv` pour les agences existantes et `site_candidates_full.csv` pour les sites potentiels. L'ensemble est complété par le présent rapport de stage et une documentation technique détaillée.

2.2 Exigences Fonctionnelles

Les exigences fonctionnelles décrivent ce que le système doit faire du point de vue de l'utilisateur. Elles ont été identifiées lors de la phase de cadrage et affinées au cours des sprints.

TABLE 2.1 – Tableau des exigences fonctionnelles

Fonctionnalité	Exigences Fonctionnelles
Gestion des données POI	Charger des fichiers POI provenant de trois sources (OSM, Google Maps, Overture Maps) Fusionner les fichiers et éliminer les doublons basés sur latitude/longitude Classer chaque POI dans une catégorie (commercial, bureau, résidentiel, transit, concurrent, agence propre) Produire un fichier merged_data_clean.csv consolidé
Scoring des agences existantes	Calculer sept scores normalisés : commercial, bureaux, résidentiel, accessibilité, concurrence, cannibalisation, densité de population Calculer un score composite pondéré à partir des sept scores Attribuer une recommandation (KEEP, MONITOR, REVIEW, CLOSE) selon des seuils prédéfinis Regrouper les agences en 5 clusters via K-Means et étiqueter chaque cluster Produire un fichier branches_scored.csv avec scores et recommandations

Sélection de nouveaux sites	Générer une grille de points candidats avec espacement configurable (par défaut 0,7 km) Filtrer les points en mer ou zones inhabitées via validation de côte stricte Calculer un score POI par pondération inverse de la distance Identifier les 10% meilleures agences par cluster comme référence Calculer la similarité cosinus entre chaque candidat et les agences benchmark Évaluer la densité de population via l'association à la ville la plus proche Calculer un score combiné paramétrable : $W_{POI} \times POI + W_{SIM} \times similarit + W_{POP} \times densitdepopulation$ Sélectionner le top 20 avec contraintes (max 2/ville, min 0,7 km entre sites) Entraîner dynamiquement un modèle Random Forest sur les agences existantes pour prédire le revenu mensuel en MAD de chaque site candidat
Visualisation et tableau de bord	Afficher une carte interactive avec agences colorées selon leur recommandation Afficher les 20 sites recommandés avec marqueurs distinctifs Proposer des filtres par ville, action, cluster et score composite Afficher des graphiques (barres, camemberts, histogrammes, nuages de points) Permettre le téléchargement des résultats au format CSV Inclure un onglet d'exécution interactive avec upload et paramètres configurables
Exécution interactive du modèle	Permettre l'upload de quatre fichiers CSV requis (incluant la densité de population) Permettre la configuration des paramètres (clusters, poids POI, poids population et poids similarité, espacement, contraintes) Afficher une barre de progression pendant l'exécution Afficher les résultats avec carte, graphiques et statistiques résumées Permettre le téléchargement (top 20 CSV, tous candidats CSV, métriques JSON)

2.2.1 Optimisation du Réseau et Auto-ML (Matrice BCG)

Dans un souci d'alignement avec les standards de l'industrie (tels que ceux prônés par les grands cabinets de conseil en stratégie), une fonctionnalité avancée d'**Optimisation du Réseau** a été intégrée sous la forme d'un pipeline *Auto-ML*.

Ce pipeline repose sur trois piliers :

1. **Analyse des Écarts (Gap Analysis) par XGBoost :** Le système entraîne de manière dynamique un modèle de régression XGBoost pour prédire le revenu "attendu" d'une agence en fonction uniquement de son potentiel géographique (densité de population

et scores POI). En comparant ce revenu attendu au revenu réel, le système calcule un "Ratio de Performance", permettant d'isoler le potentiel de l'emplacement de l'exécution managériale.

2. **Matrice Stratégique BCG :** Les agences sont ensuite projetées sur une matrice interactive (Potentiel vs Exécution) et catégorisées automatiquement :
   — **FLAGSHIP** (Ratio > 1.15, Fort Potentiel) : Agences d'excellence.
   — **REVIEW / INVEST** (Ratio < 0.85, Fort Potentiel) : Emplacement stratégique, mais sous-performance opérationnelle nécessitant un audit.
   — **MONITOR** (Ratio > 1.15, Faible Potentiel) : Bonne exécution malgré une localisation limitante.
   — **CLOSE** (Ratio < 0.85, Faible Potentiel) : Agences non rentables à fermer.
3. **Fusions Géospatiales Intelligentes :** Pour éviter la perte de parts de marché, le système utilise l'algorithme `cKDTree` de la bibliothèque SciPy. Pour chaque agence marquée comme *CLOSE* ou *REVIEW*, le système vérifie s'il existe une agence *FLAGSHIP* dans un rayon de 1,2 km. Si c'est le cas, la recommandation est intelligemment convertie en **MERGE** (Fusion), avec un traçage visuel sur la carte Folium.

2.3 Exigences Non Fonctionnelles

Les exigences non fonctionnelles définissent les qualités que le système doit posséder, indépendamment de ses fonctionnalités.

TABLE 2.2 – Tableau des exigences non fonctionnelles

Domaine	Exigences Techniques
Performance	Scoring des 1 195 agences en moins de 30 secondes Sélection de sites parmi 2 millions de points en moins de 5 minutes Chargement du dashboard et visualisations en moins de 10 secondes Exécution interactive du modèle en moins de 5 minutes
Fiabilité et Robustesse	Gestion des données manquantes sans plantage (valeurs par défaut, imputation) Détection et suppression des doublons de POI Validation que les points candidats sont sur le territoire marocain Messages d'erreur clairs avec trace complète en cas d'échec
Utilisabilité	Interface intuitive ne nécessitant aucune formation technique Visualisations claires, lisibles et informatives Paramètres configurables accompagnés de descriptions et valeurs par défaut Design professionnel cohérent avec l'image d'Attijariwafa Bank

Maintenabilité et Évolutivité	Code modulaire et documenté pour faciliter les mises à jour Architecture permettant l'ajout facile de nouvelles sources de POI Algorithmes paramétrables pour s'adapter à différents contextes Support pour l'intégration future de données socio-économiques externes
Sécurité et Confidentialité	Données traitées restant dans l'environnement sécurisé d'Attijariwafa Bank Aucune exposition de données sensibles en dehors du réseau interne Fichiers uploadés traités en mémoire sans persistance non contrôlée

2.4 Modélisation UML

La modélisation UML (Unified Modeling Language) a été utilisée pour concevoir et documenter le système de manière visuelle et structurée. UML est un langage de modélisation standardisé qui permet de représenter les différentes facettes d'un système logiciel à l'aide de diagrammes.

2.4.1 Choix d'UML pour la Modélisation

Plusieurs raisons ont motivé le choix d'UML comme outil de modélisation pour ce projet. UML constitue un langage universellement reconnu dans l'industrie du logiciel, ce qui facilite grandement la communication entre les membres de l'équipe de développement et les différentes parties prenantes. Sa polyvalence offre une variété de diagrammes permettant de modéliser les différents aspects du système, qu'il s'agisse des aspects structurels comme les classes et les composants, ou des aspects comportementaux comme les cas d'utilisation et les séquences.

Dans le cadre spécifique de ce projet, nous avons principalement utilisé trois types de diagrammes :

— **Diagramme des cas d'utilisation :** Employé pour représenter les interactions entre les différents acteurs du système et les fonctionnalités offertes par la plateforme, permettant ainsi de visualiser qui fait quoi ;

— **Diagramme de séquence :** Utilisé pour détailler le déroulement temporel des traitements dans les pipelines de scoring et de sélection de sites, illustrant la chronologie précise des opérations et les échanges entre les composants ;

— **Diagramme d'activité :** Mobilisé pour illustrer le flux de données et les étapes successives de traitement ; ce diagramme est présenté en fin de chapitre et détaille notamment la nouvelle logique d'optimisation (Auto-ML).

La visualisation claire fournie par ces diagrammes complète la documentation textuelle et rend le système beaucoup plus compréhensible pour l'ensemble des intervenants, tout en facilitant la traçabilité des exigences jusqu'à leur implémentation technique concrète.

2.5 Diagramme des Cas d'Utilisation

Le diagramme des cas d'utilisation présente les interactions entre les acteurs du système et les fonctionnalités offertes par la plateforme.

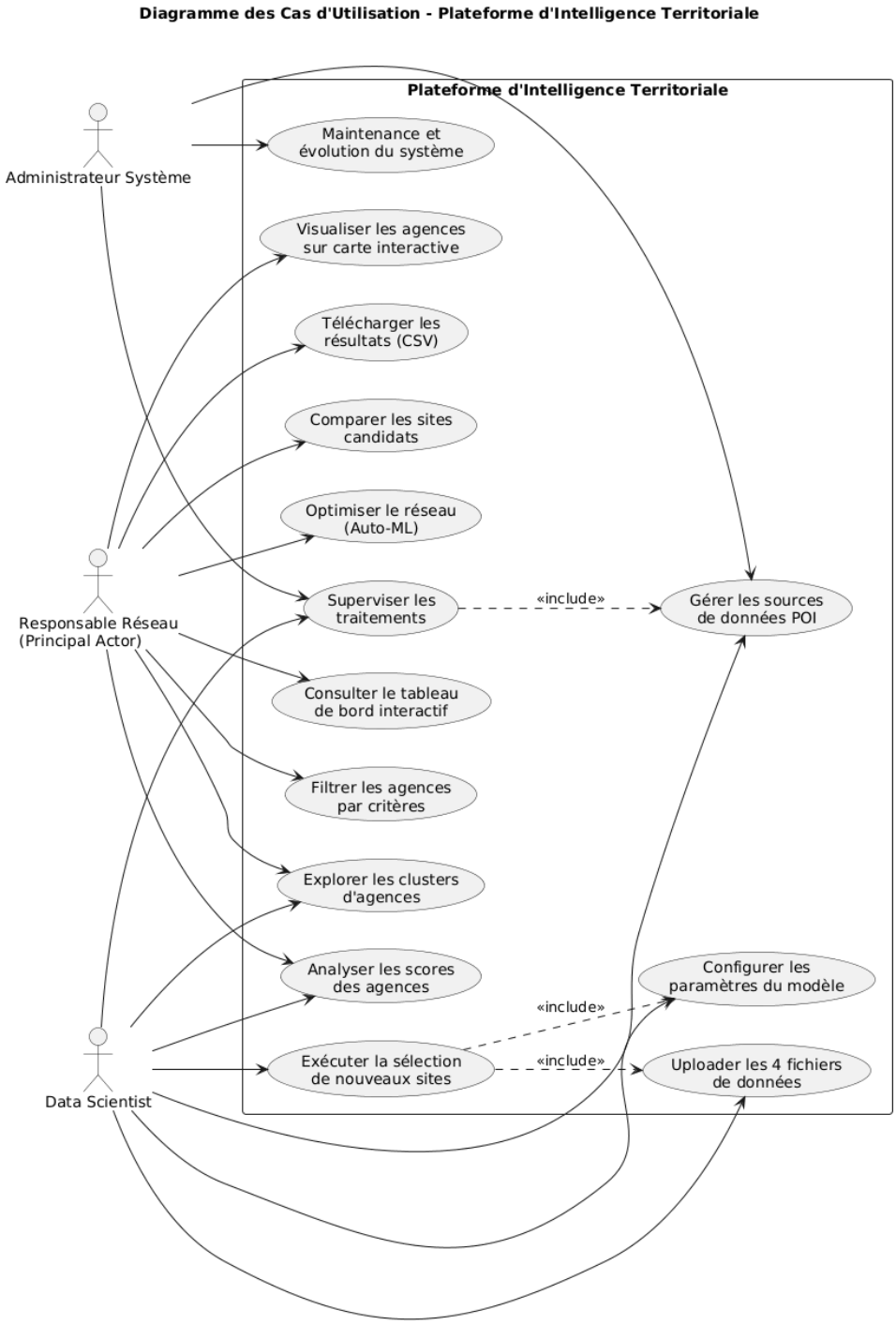


FIGURE 2.1 – Diagramme des cas d'utilisation

2.5.1 Description des Acteurs

Le système comporte trois acteurs principaux aux rôles distincts et complémentaires, chacun intervenant à des niveaux différents du processus de prise de décision.

Le **Responsable Réseau** constitue l'utilisateur principal de la plateforme. Ses activités couvrent plusieurs aspects clés de l'analyse du réseau d'agences :

— Consultation du tableau de bord interactif pour une vue d'ensemble des performances ;

— Analyse détaillée des scores attribués à chaque agence et de leurs composantes ;

— Exploration des clusters formés par les agences similaires ;

— Visualisation des recommandations directement sur la carte géographique ;

— Téléchargement des résultats pour appuyer les décisions stratégiques d'expansion ou de restructuration.

Le **Data Scientist** endosse la responsabilité de l'exécution des modèles de sélection de nouveaux sites. Ses interventions portent sur :

— Configuration des paramètres algorithmiques (nombre de clusters, poids POI/population/similarité, espacement de la grille) ;

— Upload des fichiers de données sources requis pour l'exécution du modèle ;

— Supervision des traitements pour garantir la qualité et la fiabilité des résultats produits.

Enfin, l'**Administrateur Système** assure le bon fonctionnement global de la plateforme. Il est responsable de :

— La gestion des sources de données POI et le maintien des connexions aux différentes API ;

— La supervision de l'exécution des pipelines en surveillant les logs et les performances ;

— La maintenance évolutive du système et l'intégration de nouvelles fonctionnalités.

2.5.2 Description des Cas d'Utilisation Principaux

TABLE 2.3 – Description détaillée des cas d'utilisation

Cas d'Utilisation	Description
UC1 : Consulter le tableau de bord interactif	L'utilisateur accède à la plateforme Streamlit qui affiche les KPIs globaux, les graphiques de performance et les visualisations des données
UC2 : Filtrer les agences par critères	L'utilisateur applique des filtres sur la ville, l'action recommandée, le cluster et le score composite pour affiner l'analyse
UC3 : Visualiser les agences sur carte interactive	L'utilisateur consulte la carte Folium avec les marqueurs colorés selon les recommandations et peut afficher les popups d'information
UC4 : Télécharger les résultats (CSV)	L'utilisateur exporte les données affichées (agences scorées, sites recommandés, métriques) au format CSV pour analyse externe
UC5 : Exécuter la sélection de nouveaux sites	L'utilisateur lance le pipeline complet de sélection de sites depuis le dashboard avec upload de fichiers et paramètres configurables

UC6 : Configurer les paramètres du modèle	L'utilisateur ajuste le nombre de clusters, les poids POI/population/similarité, l'espacement de grille et les contraintes de diversité géographique
UC7 : Uploader les fichiers de données	L'utilisateur charge les quatre fichiers CSV requis (branches_scored.csv, branch_performance.csv, merged_data_clean.csv, density.csv)
UC8 : Gérer les sources de données POI	L'administrateur configure et maintient les connexions aux sources OSM, Google Maps et Overture Maps pour la collecte des POI
UC9 : Superviser les traitements	L'administrateur surveille l'exécution des pipelines, vérifie les logs et intervient en cas d'erreur ou de performance dégradée
UC10 : Maintenance et évolution du système	L'administrateur met à jour le code, ajoute de nouvelles fonctionnalités et intègre de nouvelles sources de données
UC11 : Explorer les clusters d'agences	L'utilisateur analyse la répartition des agences en 5 clusters (High Performer, Commercial Hub, etc.) et visualise les caractéristiques de chaque groupe
UC12 : Analyser les scores des agences	L'utilisateur examine les sept scores normalisés et le score composite de chaque agence pour comprendre sa performance relative
UC13 : Comparer les sites candidats	L'utilisateur compare les sites recommandés selon le score combiné, le revenu prédit et la similarité aux meilleures agences
UC14 : Optimiser le réseau (Auto-ML)	L'utilisateur exécute un modèle XGBoost dynamique pour effectuer une Gap Analysis et catégoriser les agences sur une Matrice BCG stratégique (Flagship, Review, Monitor, Close, Merge)

2.6 Diagramme de Séquence

Le diagramme de séquence détaille l'ordre temporel des interactions entre l'utilisateur, le tableau de bord (dashboard) et les pipelines de traitement lors de l'exécution de la sélection de nouveaux sites, qui est la fonctionnalité la plus complexe du système.

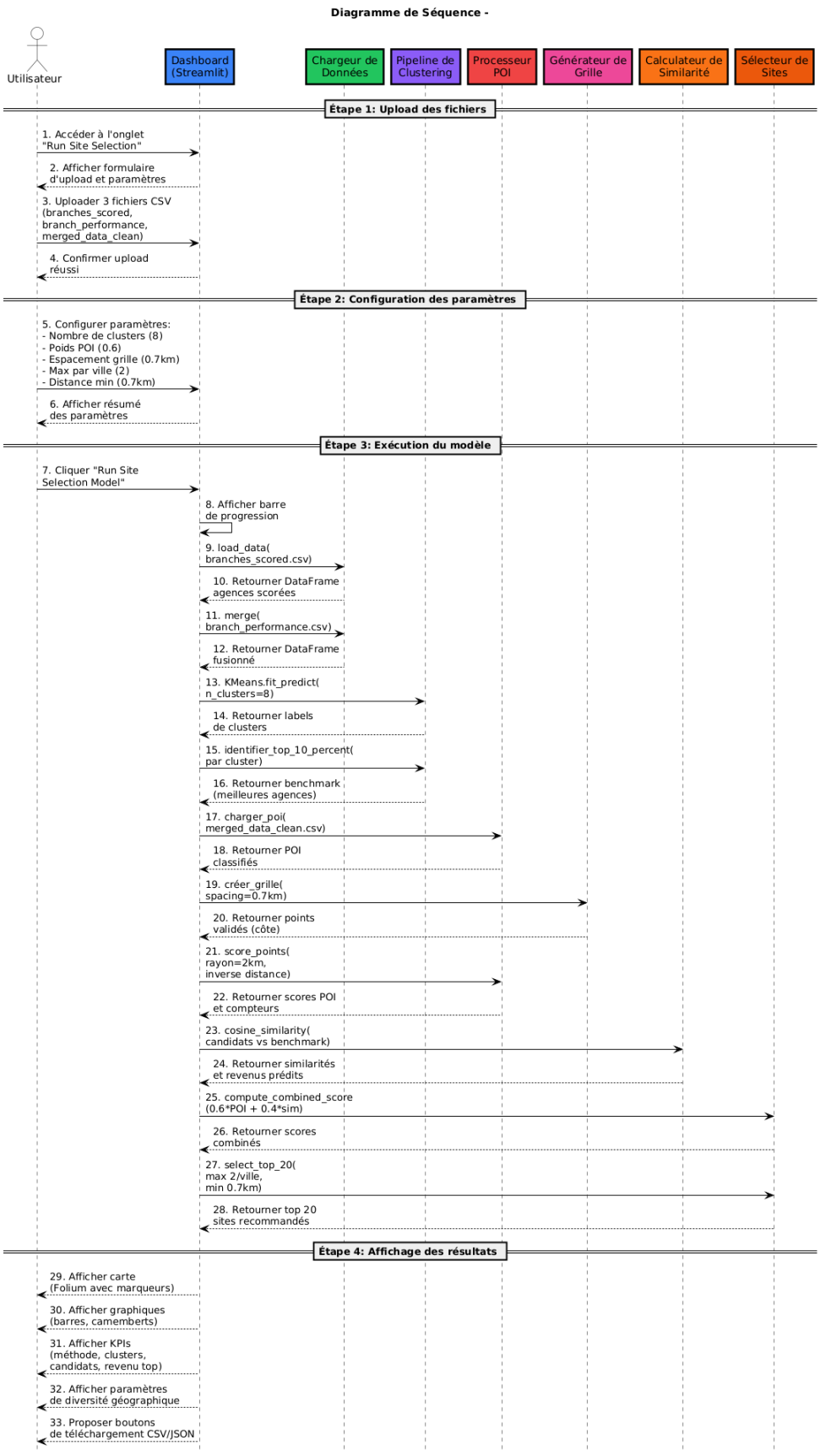


FIGURE 2.2 – Diagramme de séquence

2.6.1 Description Détaillée des Étapes

Le diagramme de séquence présenté ci-dessus illustre le workflow complet de l'exécution de la sélection de nouveaux sites, depuis l'initialisation par l'utilisateur jusqu'à l'affichage des résultats finaux. Ce processus se décompose en six phases successives, chacune jouant un rôle essentiel dans la chaîne de traitement.

Phase 1 : Initialisation

Cette première phase correspond aux premiers échanges entre l'utilisateur et le système. L'utilisateur navigue vers l'onglet dédié à la sélection de sites au sein du tableau de bord, où le système affiche le formulaire d'upload avec les quatre champs requis pour les fichiers de données (données existantes, performances, POIs et densités par ville) ainsi que les paramètres configurables disponibles. L'utilisateur procède alors au téléchargement des fichiers CSV nécessaires, et le système valide la conformité de ces fichiers avant de confirmer le succès de l'opération.

Phase 2 : Configuration des Paramètres

Cette phase voit l'utilisateur ajuster les différents paramètres du modèle à l'aide des curseurs et cases à cocher mis à sa disposition dans l'interface. Les paramètres configurables incluent :

— Le **nombre de clusters** pour l'algorithme K-Means (par défaut 8) ;

— Les **poids relatifs** attribués au score POI et à la densité de population dans le calcul combiné (le poids de la similarité se déduisant de ceux-ci) ;

— L'**espacement** entre les points de la grille de candidats (par défaut 0,7 km) ;

— Les **contraintes de diversité géographique** limitant le nombre d'agences par ville et la distance minimale entre sites sélectionnés.

Le système affiche ensuite un résumé synthétique de l'ensemble des paramètres pour validation finale avant l'exécution, permettant de personnaliser l'exécution selon les besoins stratégiques du moment.

Phase 3 : Traitement Initial des Données

Le dashboard charge les données des agences existantes depuis le fichier `branches_scored.csv`, puis les fusionne avec les données de performance issues du fichier `branch_performance.csv` en utilisant le nom de l'agence comme clé de jointure. La densité de population est ensuite rattachée à chaque agence existante via une requête spatiale K-d tree sur le fichier `density.csv`, identifiant pour chaque agence la ville la plus proche et récupérant sa population. Le pipeline de clustering applique ensuite l'algorithme K-Means avec le nombre de clusters spécifié par l'utilisateur sur les caractéristiques de scoring des agences. Une fois le clustering terminé, les dix pour cent meilleures agences de chaque cluster sont identifiées en se basant sur leur revenu mensuel pour former le jeu de benchmark. Ces agences d'excellence serviront de référence pour le calcul ultérieur de similarité cosinus, représentant le profil idéal que les nouveaux sites candidats devraient approcher.

Phase 4 : Processing POI et Génération de la Grille

Cette phase concerne le traitement des points d'intérêt et la création de la grille de candidats potentiels. Le processeur POI charge le fichier `merged_data_clean.csv` et procède à la classification de chaque point d'intérêt dans l'une des six catégories définies : commercial, bureau, résidentiel, transit, concurrent ou agence propre du réseau Attijariwafa. Parallèlement, le générateur de grille crée des points candidats couvrant l'intégralité du territoire marocain selon l'espacement configuré par l'utilisateur.

Plusieurs filtres sont ensuite appliqués pour garantir la qualité des candidats :

— **Validation de côte stricte :** Élimination des points situés en mer ou dans des zones désertiques inhabitées ;

— **Filtrage de proximité :** Écartement des points à moins de 700 mètres d'une agence existante pour éviter les redondances géographiques ;

— **Filtrage par densité POI :** Suppression des candidats avec moins de 5 POI dans un rayon de 2 km.

Le scoreur calcule enfin un score POI pour chaque point candidat survivant en utilisant une pondération inversement proportionnelle à la distance dans un rayon de recherche de deux kilomètres. À cette étape, la métrique de densité de population est également récupérée pour chaque candidat en identifiant la ville la plus proche (à partir des données fournies dans le fichier de densité) via un modèle de K-d tree, puis cette métrique est normalisée sur l'ensemble de la grille.

Phase 5 : Prédiction de Revenu, Similarité et Sélection Finale

Cette phase constitue le cœur algorithmique du processus et se décompose en deux étapes distinctes.

Dans un premier temps, le système entraîne dynamiquement un modèle de régression **Random Forest** (`RandomForestRegressor` de scikit-learn, configuré avec 100 arbres de décision et une profondeur maximale de 6) sur les données des agences existantes. Les sept features d'entraînement sont les six scores POI normalisés (commercial, bureau, résidentiel, accessibilité, concurrence, cannibalisation) auxquels s'ajoute la densité de population. La variable cible est le revenu mensuel en MAD. Les valeurs aberrantes sont exclues (percentiles 1% et 99%) et les features sont standardisées via un `StandardScaler` avant l'entraînement. Ce choix de modèle a été validé par un processus de benchmarking rigoureux documenté dans le notebook `model_benchmarking.ipynb`, où le Random Forest a obtenu le meilleur coefficient de détermination ($R^2 = 0.88$) en validation croisée à 5 plis, devançant le Ridge ($R^2 = 0.88$), le Gradient Boosting ($R^2 = 0.87$) et le XGBoost ($R^2 = 0.86$). Le modèle entraîné est ensuite appliqué à l'ensemble des candidats de la grille pour produire la prédiction de revenu mensuel (`predicted_revenue_mad`) de chaque site.

Dans un second temps, le calculateur de similarité détermine la similarité cosinus entre le profil POI de chaque candidat et les profils des agences benchmark identifiées précédemment. Cette métrique de similarité (`similarity_to_best`) est utilisée comme composante du score combiné, et non pour la prédiction de revenu.

Le sélecteur calcule ensuite le score combiné paramétrable selon la formule :

$$\textbf{Score Combiné} = W_{POI} \times ScorePOI + W_{SIM} \times Similarity + W_{POP} \times ScoreDensitPopulation$$

Puis procède à la sélection du top 20 des emplacements en appliquant rigoureusement les contraintes de diversité géographique configurées précédemment, à savoir un maximum de deux agences par ville et une distance minimale de sept cents mètres entre chaque site sélectionné.

Phase 6 : Visualisation des Résultats

La dernière phase correspond à l'affichage structuré de l'ensemble des résultats obtenus. Le dashboard présente plusieurs éléments clés :

— **Carte interactive Folium :** Les vingt sites recommandés sous forme de marqueurs en forme d'étoile de couleur orange, superposés aux agences existantes du réseau ;

— **Graphiques analytiques Plotly :** Barres horizontales des revenus prédits, scores de similarité cosinus, distribution des types de POI, et histogramme de la répartition des revenus ;

— **Indicateurs clés de performance :** Modèle de prédiction employé (Random Forest), nombre de clusters utilisé, nombre total de candidats traités et revenu prédit pour le site classé premier ;

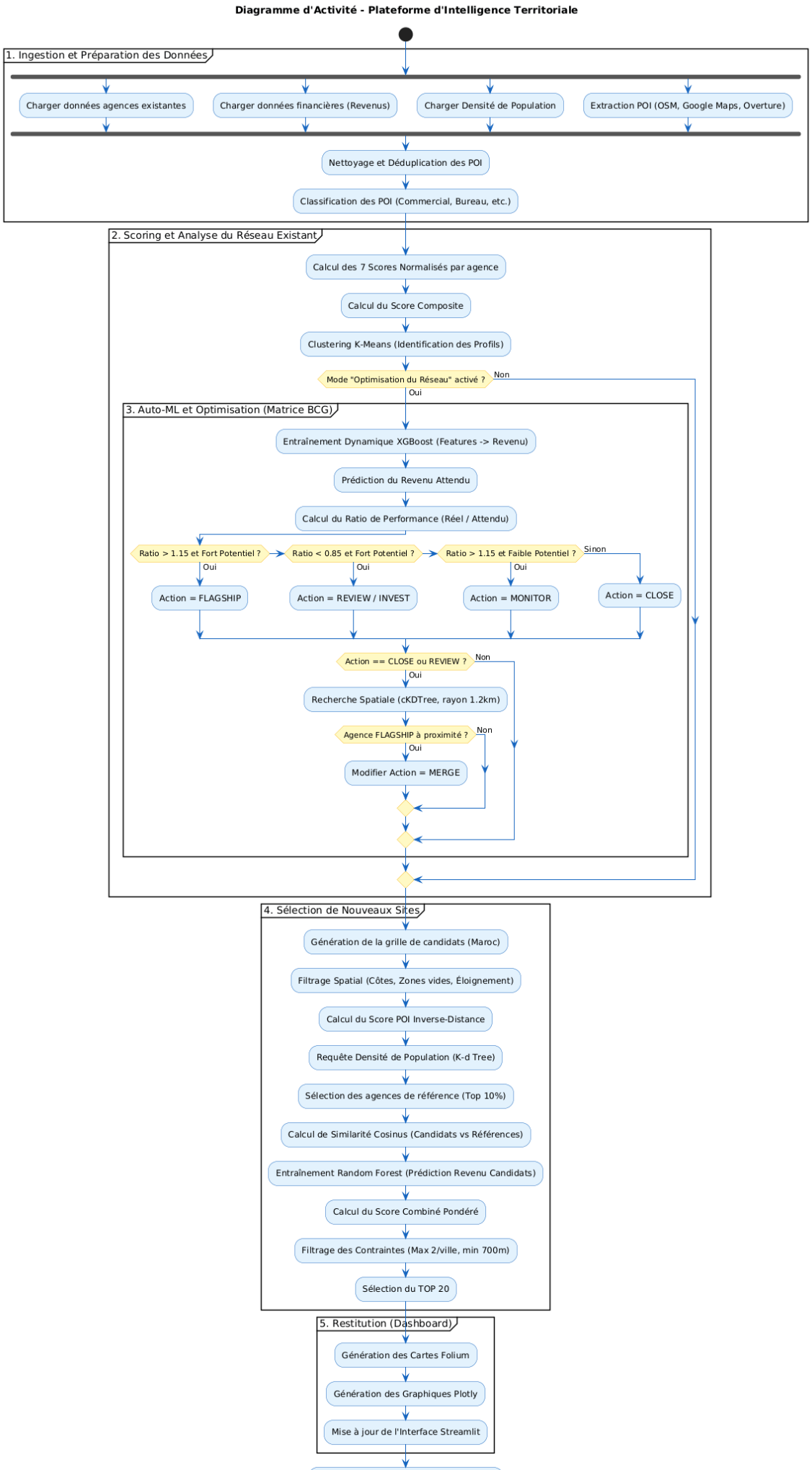
— **Boutons de téléchargement :** Export des résultats en trois formats : CSV pour les 20 sites recommandés, CSV pour l'ensemble des candidats évalués, et JSON pour les métriques détaillées du modèle.

2.6.2 Diagramme de Séquence Alternatif : Scoring des Agences

Un second diagramme de séquence, bien que non inclus dans ce rapport, est disponible dans la documentation technique du projet. Il détaille le processus complet de scoring des agences existantes. Ce workflow comprend successivement le chargement des données depuis le fichier `branches_scored.csv`, le calcul des sept scores normalisés que sont les scores commercial, bureau, résidentiel, accessibilité, concurrence, cannibalisation et densité de population, puis le calcul du score composite pondéré à partir de ces sept dimensions. S'ensuivent la classification des agences en quatre catégories de recommandation que sont KEEP, MONITOR, REVIEW et CLOSE selon des seuils prédéfinis, le regroupement des agences en cinq clusters via l'algorithme K-Means, et enfin la production du fichier de résultats `branches_scored.csv` enrichi de l'ensemble des scores et recommandations calculés.

2.7 Diagramme d'Activité

Le diagramme d'activité illustre le flux de données global de la plateforme, depuis l'ingestion des sources hétérogènes (POI, performances financières, démographie) jusqu'à la sélection finale de nouveaux sites et la restitution sur le tableau de bord. Il met particulièrement en évidence la nouvelle logique d'optimisation du réseau (Auto-ML) impliquant l'entraînement dynamique d'un modèle XGBoost, le calcul du ratio de performance (Gap Analysis) et l'application des règles spatiales (cKDTree) pour détecter les fusions (Mergers).


Diagramme d'Activité - Plateforme d'Intelligence Territoriale
1. Ingestion et Préparation des Données
Charger données agences existantes
Charger données financières (Revenus)
Charger Densité de Population
Extraction POI (OSM, Google Maps, Overture)
Nettoyage et Déduplication des POI
Classification des POI (Commercial, Bureau, etc.)
2. Scoring et Analyse du Réseau Existant
Calcul des 7 Scores Normalisés par agence
Calcul du Score Composite
Clustering K-Means (Identification des Profils)
Mode "Optimisation du Réseau" activé ?
Non
Oui
3. Auto-ML et Optimisation (Matrice BCG)
Entraînement Dynamique XGBoost (Features -> Revenu)
Prédiction du Revenu Attendu
Calcul du Ratio de Performance (Réel / Attendu)
Ratio > 1.15 et Fort Potentiel ?
Oui
Action = FLAGSHIP
Ratio < 0.85 et Fort Potentiel ?
Oui
Action = REVIEW / INVEST
Ratio > 1.15 et Faible Potentiel ?
Oui
Action = MONITOR
Sinon
Action = CLOSE
Action == CLOSE ou REVIEW ?
Non
Oui
Recherche Spatiale (cKDTree, rayon 1.2km)
Agence FLAGSHIP à proximité ?
Non
Oui
Modifier Action = MERGE
4. Sélection de Nouveaux Sites
Génération de la grille de candidats (Maroc)
Filtrage Spatial (Côtes, Zones vides, Éloignement)
Calcul du Score POI Inverse-Distance
Requête Densité de Population (K-d Tree)
Sélection des agences de référence (Top 10%)
Calcul de Similarité Cosinus (Candidats vs Références)
Entraînement Random Forest (Prédiction Revenu Candidats)
Calcul du Score Combiné Pondéré
Filtrage des Contraintes (Max 2/ville, min 700m)
Sélection du TOP 20
5. Restitution (Dashboard)
Génération des Cartes Folium
Génération des Graphiques Plotly
Mise à jour de l'Interface Streamlit

Chapitre 3

Cadre Théorique et Fondements Mathématiques

Ce chapitre constitue le socle théorique de l'ensemble du projet. L'ingénierie des données et la modélisation prédictive exigent une compréhension rigoureuse des mécanismes mathématiques sous-jacents aux algorithmes employés. Ainsi, ce chapitre dépasse la simple abstraction algorithmique pour plonger au cœur de l'optimisation, du calcul différentiel, de l'algèbre linéaire et de la théorie probabiliste qui régissent les modèles d'apprentissage automatique utilisés pour la sélection de sites et l'optimisation du réseau.

3.1 Fondements Théoriques de l'Apprentissage Automatique

L'apprentissage automatique (Machine Learning) repose sur la capacité d'un système à inférer des règles mathématiques complexes à partir de données empiriques $\mathcal{D}$, dans le but de généraliser ces règles à de nouvelles instances invisibles.

3.1.1 Paradigmes d'Apprentissage

Les algorithmes mobilisés dans ce projet s'inscrivent dans deux paradigmes distincts mais complémentaires :

— **L'apprentissage non supervisé :** Dans ce cadre, l'ensemble de données $\mathcal{D} = \{x_1, x_2, \ldots, x_N\}$ est dépourvu d'étiquettes. L'objectif est d'identifier la topologie sous-jacente ou la structure latente de l'espace des caractéristiques R^d. Nous mobilisons ces algorithmes (K-Means, DBSCAN, PCA) pour segmenter le réseau existant, identifier les zones géographiques denses, et réduire la dimensionnalité de l'espace de représentation.

— **L'apprentissage supervisé :** L'ensemble de données est constitué de paires $\mathcal{D} = \{(x_1, y_1), \ldots, (x_N, y_N)\}$ où $x_i \in R^d$ est le vecteur de caractéristiques (les scores POI, la densité) et $y_i \in R$ est la cible continue (le revenu financier). L'algorithme cherche à approximer une fonction inconnue $F : R^d \to R$ minimisant l'espérance mathématique d'une fonction de perte.

3.1.2 Le Compromis Biais-Variance

L'un des défis fondamentaux de la modélisation prédictive est le compromis Biais-Variance (Bias-Variance Tradeoff). L'erreur de généralisation attendue $Err(x)$ d'un modèle $\hat{f}(x)$ pour un point x donné, face à un véritable signal $f(x)$ bruité par $\epsilon \sim \mathcal{N}(0, \sigma^2)$, se décompose mathématiquement ainsi :

$$Err(x) = E\left[(y - \hat{f}(x))^2\right] \tag{3.1}$$

En développant cette espérance, le théorème de décomposition prouve que :

$$Err(x) = \Big(E[\hat{f}(x)] - f(x)\Big)^2 + E\left[(\hat{f}(x) - E[\hat{f}(x)])^2\right] + \sigma^2 \tag{3.2}$$

Ce qui se traduit par :

$$Err(x) = Biais(\hat{f}(x))^2 + Var(\hat{f}(x)) + BruitIrrductible \tag{3.3}$$

— Le **Biais** quantifie l'erreur due à des hypothèses simplistes du modèle (ex : régression linéaire sur des données non linéaires), conduisant au sous-apprentissage (Underfitting).

— La **Variance** quantifie la sensibilité du modèle aux fluctuations de l'ensemble d'entraînement. Un modèle à forte variance modélise le bruit géospatial aléatoire, conduisant au surapprentissage (Overfitting).

Tout l'enjeu du paramétrage des modèles (comme la profondeur des arbres XGBoost) consiste à trouver le point d'inflexion minimisant la somme de ces deux erreurs.

3.1.3 Surapprentissage et Validation Croisée (K-Fold)

Pour estimer de manière non biaisée l'erreur de généralisation et éviter le surapprentissage, la méthode de la validation croisée à K-plis (K-Fold Cross-Validation) est une norme académique stricte.

L'ensemble de données $\mathcal{D}$ est partitionné aléatoirement en K sous-ensembles mutuellement exclusifs $D_1, D_2, \ldots, D_K$ de tailles approximativement égales. Le modèle est entraîné sur $K-1$ sous-ensembles et évalué sur le sous-ensemble restant D_k. Ce processus est itéré K fois. L'erreur estimée par validation croisée, $CV_{(K)}$, est la moyenne arithmétique des erreurs de chaque pli :

$$CV_{(K)} = \frac{1}{K} \sum_{k=1}^{K} L_k(y, \hat{f}^{-k}(x)) \tag{3.4}$$

où $\hat{f}^{-k}$ est le modèle entraîné sans le sous-ensemble k. Dans notre modélisation financière, l'utilisation systématique de cette méthode garantit que nos prédictions de revenus pour de nouvelles agences ne souffrent pas de mémorisation des données d'entraînement.

3.2 Théorie du Prétraitement et Qualité des Données

La robustesse des algorithmes spatiaux dépend intrinsèquement de la distribution statistique des données en entrée. Le prétraitement n'est donc pas une étape heuristique, mais repose sur des transformations mathématiques strictes.

3.2.1 Standardisation Statistique (Z-Score)

La standardisation est impérative lorsque les caractéristiques n'évoluent pas dans les mêmes échelles de grandeur (par exemple, la distance en mètres vs le nombre de banques concurrentes). L'absence de standardisation perturbe sévèrement les fonctions de distance Euclidienne (utilisées par K-Means et cKDTree) et ralentit la convergence des algorithmes basés sur la descente de gradient.

La transformation Z-score centre la distribution autour de zéro et réduit son écart-type à l'unité. Pour une variable aléatoire X_j d'espérance μ_j et d'écart-type σ_j, la variable standardisée Z_j est définie par :

$$Z_{ij} = \frac{x_{ij} - \mu_j}{\sigma_j} \tag{3.5}$$

Ainsi, $E[Z_j] = 0$ et $Var(Z_j) = 1$. Cette transformation isométrique préserve la corrélation entre les variables tout en stabilisant le conditionnement des matrices lors de l'apprentissage.

3.2.2 Traitement Statistique des Valeurs Aberrantes (Outliers)

Les valeurs aberrantes financières (agences générant des revenus statistiquement anormaux) déforment la fonction de perte L_2, qui pénalise ces écarts quadratiquement. Pour identifier ces valeurs de manière robuste (c'est-à-dire sans être influencé par la moyenne qui est elle-même sensible aux extrêmes), la méthode de l'Écart Interquartile (Interquartile Range - IQR) est privilégiée.

Soient Q_1 et Q_3 les premier et troisième quartiles de la distribution des revenus Y. L'IQR est défini comme la dispersion de la moitié centrale des données :

$$IQR = Q_3 - Q_1 \tag{3.6}$$

Une observation y_i est considérée comme aberrante si elle n'appartient pas à l'intervalle de confiance défini par le paramètre de sévérité τ (généralement $\tau = 1.5$) :

$$y_i \notin [Q_1 - \tau \cdot IQR, \quad Q_3 + \tau \cdot IQR] \tag{3.7}$$

Les observations en dehors de ces bornes (Tukey's fences) sont soit écrêtées (Clipping/Winsorisation) soit exclues pour préserver la stabilité asymptotique du modèle de régression.

3.3 Apprentissage Non Supervisé et Modélisation Géospatiale

3.3.1 K-Means et Optimisation de Lloyd

L'algorithme K-Means cherche à partitionner l'espace R^d en K sous-ensembles $S = \{S_1, S_2, \ldots, S_K\}$ en minimisant la variance intra-cluster (Inertie). Mathématiquement, il s'agit de la résolution d'un problème d'optimisation non convexe :

$$\arg\min_{S} \sum_{k=1}^{K} \sum_{x_i \in S_k} ||x_i - \mu_k||_2^2 \tag{3.8}$$

où μ_k est le centroïde du cluster S_k. L'algorithme de Lloyd résout ce problème de manière itérative. La démonstration stipulant que le centroïde optimal est strictement la moyenne arithmétique s'obtient en annulant le gradient de la fonction de coût J par rapport à μ_k :

$$\nabla_{\mu_k} J = \nabla_{\mu_k} \sum_{x_i \in S_k} (x_i - \mu_k)^T (x_i - \mu_k) = -2 \sum_{x_i \in S_k} (x_i - \mu_k) = 0 \tag{3.9}$$

En isolant μ_k, nous obtenons l'équation de mise à jour formelle :

$$\mu_k = \frac{1}{|S_k|} \sum_{x_i \in S_k} x_i \tag{3.10}$$

3.3.2 Initialisation Probabiliste : K-Means++

L'algorithme de Lloyd est sensible à l'initialisation des centroïdes et peut converger vers des minimums locaux sous-optimaux. Pour y remédier, l'algorithme K-Means++ (Arthur et Vassilvitskii, 2007) propose une initialisation probabiliste. Le premier centroïde c_1 est choisi uniformément au hasard parmi les points de données. Ensuite, pour chaque point x_i, on calcule la distance $D(x_i)$ par rapport au centroïde le plus proche déjà choisi. Le prochain centroïde est tiré avec une probabilité proportionnelle au carré de cette distance :

$$P(x_i = c_{nouveau}) = \frac{D(x_i)^2}{\sum_{j=1}^{N} D(x_j)^2} \tag{3.11}$$

Cette approche écarte algorithmiquement les centroïdes initiaux, garantissant une approximation en espérance de l'ordre de $O(\log K)$ du minimum global de l'inertie.

3.3.3 Évaluation du Clustering : Le Score de Silhouette

Puisque nous ne disposons pas de vérité terrain (Ground Truth) pour valider la segmentation des agences, des métriques de validation interne doivent être appliquées. Le Coefficient de Silhouette évalue simultanément la compacité d'un cluster et sa séparation vis-à-vis des autres.

Pour une observation $i \in S_k$, on définit : - $a(i)$: la distance moyenne intra-cluster (cohésion). - $b(i)$: la distance moyenne au cluster le plus proche S_m avec $m \neq k$ (séparation).

Le score de silhouette $s(i)$ pour le point i s'écrit :

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}} \tag{3.12}$$

Le score global est l'espérance $E[s(i)]$ sur l'ensemble du jeu de données. Encadré dans $[-1, 1]$, un score proche de 1 indique une segmentation de haute qualité, tandis qu'un score négatif indique des assignations incorrectes.

3.3.4 Clustering Basé sur la Densité : DBSCAN

Alors que K-Means contraint les clusters à une forme hypersphérique (Voronoï), l'algorithme DBSCAN (Density-Based Spatial Clustering of Applications with Noise) identifie des régions de haute densité spatiale de forme arbitraire. Cette méthode est cruciale dans la modélisation urbaine pour distinguer les zones très denses des zones éparses.

DBSCAN repose sur deux hyperparamètres : un rayon ϵ et un nombre minimum de points $MinPts$. Le concept clé est l'ϵ-voisinage d'un point p, noté $N_\epsilon(p)$:

$$N_\epsilon(p) = \{q \in \mathcal{D} \mid ||p - q||_2 \leq \epsilon\} \tag{3.13}$$

Un point p est qualifié de "Point Noyau" (Core Point) si $|N_\epsilon(p)| \geq MinPts$. Un cluster se forme mathématiquement en regroupant itérativement tous les points mutuellement "directement atteignables par densité". Les points qui ne sont atteignables par aucun point noyau sont mathématiquement classifiés comme bruit (Noise).

3.3.5 PCA : Optimisation sous contrainte et Décomposition (SVD)

L'Analyse en Composantes Principales (PCA) projette les données sur un sous-espace orthogonal maximisant la variance, afin de lutter contre la malédiction de la dimensionnalité. Soit $X \in R^{N \times d}$ la matrice des données centrées et $\Sigma = \frac{1}{N-1} X^T X$ la matrice de covariance. Nous cherchons le vecteur de projection $v \in R^d$ qui maximise la variance projetée $v^T \Sigma v$, sous la stricte contrainte d'orthonormalité $v^T v = 1$.

Ce problème se résout élégamment par la méthode des multiplicateurs de Lagrange :

$$\mathcal{L}(v, \lambda) = v^T \Sigma v - \lambda(v^T v - 1) \tag{3.14}$$

En dérivant le Lagrangien par rapport au vecteur v et en l'annulant :

$$\frac{\partial \mathcal{L}}{\partial v} = 2\Sigma v - 2\lambda v = 0 \Sigma v = \lambda v \tag{3.15}$$

L'équation obtenue est l'équation caractéristique aux valeurs propres. La solution optimale pour v (la première composante principale) est le vecteur propre de la matrice de covariance associé à sa plus grande valeur propre λ_{max}. En pratique algorithmique, ce problème est résolu non pas par le calcul de la matrice de covariance, mais par la Décomposition en Valeurs Singulières (SVD) de la matrice de données $X = USV^T$, offrant une stabilité numérique largement supérieure en évitant le calcul du produit $X^T X$.

3.4 Apprentissage Supervisé : Des Modèles Linéaires aux Modèles Arborescents

3.4.1 Moindres Carrés Ordinaires (OLS) et Théorème de Gauss-Markov

La régression linéaire classique vise à trouver le vecteur de paramètres β tel que l'hyperplan $y = X\beta + \epsilon$ approxime au mieux les données. L'estimateur OLS (Ordinary Least Squares) minimise la Somme des Carrés des Résidus (RSS) :

$$RSS(\beta) = \sum_{i=1}^{N} (y_i - x_i^T \beta)^2 = (y - X\beta)^T (y - X\beta) \tag{3.16}$$

En dérivant par rapport à β, on obtient la solution analytique :

$$\hat{\beta}^{OLS} = (X^T X)^{-1} X^T y \tag{3.17}$$

Le théorème de Gauss-Markov stipule que sous l'hypothèse d'homoscédasticité et d'absence d'autocorrélation des erreurs, $\hat{\beta}^{OLS}$ est le "BLUE" (Best Linear Unbiased Estimator). Cependant, si les variables prédictives (scores POI) sont colinéaires, la matrice $X^T X$ est mal conditionnée, son déterminant tend vers zéro, et l'estimateur devient extrêmement instable avec une variance explosive.

3.4.2 Régression Ridge et Interprétation Bayésienne

La régression Ridge (Tikhonov regularization) résout le problème de colinéarité en introduisant une pénalité L_2. La nouvelle fonction de coût devient :

$$J(\beta) = ||y - X\beta||_2^2 + \alpha ||\beta||_2^2 \tag{3.18}$$

En annulant le gradient $\nabla_\beta J(\beta) = -2X^T y + 2(X^T X + \alpha I)\beta = 0$, la solution exacte est :

$$\hat{\beta}^{ridge} = (X^T X + \alpha I)^{-1} X^T y \tag{3.19}$$

L'ajout de αI (où I est la matrice identité) sur la diagonale garantit que le nouveau système est strictement défini positif et donc parfaitement inversible. D'un point de vue Bayésien, la régression Ridge équivaut à un estimateur MAP (Maximum A Posteriori) où l'on suppose que les coefficients β suivent une distribution *A Priori* Gaussienne centrée en zéro de variance τ^2, avec $\alpha = \sigma^2/\tau^2$. Cette contrainte agit comme un puits gravitationnel, tirant doucement les variables moins informatives vers zéro.

3.4.3 Arbres de Décision (CART) et Réduction d'Impureté

Face à la non-linéarité inhérente aux dynamiques territoriales, les modèles arborescents (Classification and Regression Trees - CART) segmentent récursivement l'espace R^d en régions hyper-rectangulaires R_m. Pour une tâche de régression, la valeur prédite dans chaque région R_m est simplement la moyenne des observations s'y trouvant : $\hat{c}_m = moyenne(y_i \mid x_i \in R_m)$.

L'algorithme glouton (greedy) cherche la variable de séparation j et le seuil s qui définissent les demi-plans $R_1(j,s) = \{X \mid X_j \leq s\}$ et $R_2(j,s) = \{X \mid X_j > s\}$. Le critère de scission mathématique minimise la variance intra-nœud (MSE) :

$$\arg\min_{j,s} \left[\sum_{x_i \in R_1(j,s)} (y_i - \hat{c}_1)^2 + \sum_{x_i \in R_2(j,s)} (y_i - \hat{c}_2)^2 \right] \tag{3.20}$$

Un arbre unique a cependant une variance extrêmement élevée : une infime variation des données d'entraînement modifie totalement la cascade de règles décisionnelles de l'arbre.

3.4.4 Random Forest : Bagging et Décorrélation Statistique

Le Random Forest transcende les limites des arbres simples par le principe de *Bagging* (Bootstrap Aggregating) associé à la méthode du *Feature Subsampling*. Le modèle agrège B arbres d'entraînement générés sur des tirages bootstrap (avec remise). La prédiction finale est l'espérance :

$$\hat{y} = \frac{1}{B} \sum_{b=1}^{B} f_b(x) \tag{3.21}$$

La beauté de ce modèle s'explique par la formule de la variance d'un ensemble d'arbres. Si les arbres sont identiquement distribués mais corrélés avec un facteur ρ, de variance individuelle σ^2, alors la variance de la forêt entière est :

$$Var(\hat{y}) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2 \tag{3.22}$$

Si les arbres étaient construits de manière classique, les variables POI dominantes seraient toujours choisies à la racine, rendant la corrélation ρ proche de 1 et annulant l'intérêt de la forêt (le second terme tend vers 0 mais le premier reste σ^2). En forçant l'arbre à ne chercher le meilleur "split" que parmi un sous-ensemble restreint de $m \approx p/3$ variables tirées aléatoirement, la corrélation ρ s'effondre. Le premier terme de l'équation asymptotique tend alors vers zéro, produisant un modèle d'une robustesse exceptionnelle sans augmenter le biais.

3.4.5 Gradient Boosting Machine (GBM) Classique

Contrairement au Bagging, le *Boosting* est une approche séquentielle. Le Gradient Boosting Machine (Friedman, 2001) construit l'ensemble d'arbres en minimisant la fonction de perte via une descente de gradient dans l'espace fonctionnel.

À l'itération t, le modèle global $F_t(x)$ s'écrit sous la forme additive : $F_t(x) = F_{t-1}(x) + \gamma_t h_t(x)$. L'algorithme calcule le pseudo-résidu (pseudo-residual) négatif, qui est le gradient de la perte L évalué sur les prédictions précédentes :

$$r_{it} = - \left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} \right]_{F(x)=F_{t-1}(x)} \tag{3.23}$$

Un nouvel arbre faible $h_t(x)$ est ensuite ajusté pour prédire non pas la cible y, mais les résidus r_{it}, corrigeant exactement là où les modèles précédents ont échoué.

3.4.6 L'Architecture Avancée d'Extreme Gradient Boosting (XGBoost)

XGBoost (Chen et Guestrin, 2016) est l'état de l'art absolu des algorithmes de Boosting. Il s'agit du moteur principal de notre fonctionnalité Auto-ML pour la Gap Analysis du réseau d'agences.

Contrairement au GBM classique qui utilise des heuristiques, XGBoost formule l'apprentissage d'un nouvel arbre $f_t(x)$ directement comme l'optimisation rigoureuse de la fonction objectif à l'étape t :

$$\mathcal{L}^{(t)} = \sum_{i=1}^{N} l(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)) + \Omega(f_t) \tag{3.24}$$

Pour résoudre cette équation sans exiger que la perte soit différentiable analytiquement simple, XGBoost recourt à l'approximation de Taylor au second ordre :

$$\mathcal{L}^{(t)} \simeq \sum_{i=1}^{N} \left[l(y_i, \hat{y}_i^{(t-1)}) + g_i f_t(x_i) + \frac{1}{2} h_i f_t^2(x_i) \right] + \Omega(f_t) \tag{3.25}$$

Où g_i est le Gradient (dérivée première) et h_i le Hessien (dérivée seconde de la courbure) :

$$g_i = \partial_{\hat{y}} l(y_i, \hat{y}_i^{(t-1)}) \quad et \quad h_i = \partial_{\hat{y}}^2 l(y_i, \hat{y}_i^{(t-1)}) \tag{3.26}$$

La régularisation explicite $\Omega(f_t)$, l'autre innovation majeure d'XGBoost, pénalise la complexité de la structure d'arbre. Soit T le nombre de feuilles, et w le vecteur des poids de ces feuilles.

$$\Omega(f_t) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^{T} w_j^2 \tag{3.27}$$

L'équation de Taylor devient alors la somme de fonctions quadratiques indépendantes j (une par feuille de l'arbre). L'algorithme dérive l'équation par rapport à w_j et l'annule pour trouver le poids de feuille optimal exact, connu sous le nom de pas de Newton-Raphson :

$$w_j^* = -\frac{\sum_{i \in I_j} g_i}{\sum_{i \in I_j} h_i + \lambda} \tag{3.28}$$

En remplaçant w_j^* dans l'équation, on obtient la métrique exacte mesurant la qualité (le "Gain" d'information régularisé) d'une scission S générant les nœuds Gauche (G) et Droit (D) :

$$Gain = \frac{1}{2} \left[\frac{(\sum_G g_i)^2}{\sum_G h_i + \lambda} + \frac{(\sum_D g_i)^2}{\sum_D h_i + \lambda} - \frac{(\sum_{G \cup D} g_i)^2}{\sum_{G \cup D} h_i + \lambda} \right] - \gamma \tag{3.29}$$

Ce niveau d'optimisation mathématique explique les performances foudroyantes de XGBoost pour capturer les non-linéarités complexes du revenu géospatial.

3.5 Théorie Algorithmique Spatiale et Partitionnement

La modélisation d'un réseau bancaire national nécessite l'évaluation continue de millions de requêtes de proximité spatiale (cannibalisation, identification de candidats proches).

3.5.1 L'Architecture des Arbres k-d (cKDTree)

L'arbre k-d (k-dimensional tree) partitionne un espace euclidien R^k via une série d'hyperplans sécants orthogonaux aux axes.

Construction ($O(N \log N)$) : À chaque niveau l de la structure de données récursive, l'algorithme sélectionne l'axe $a \in \{latitude, longitude\}$ présentant la plus grande variance statistique des points. Il détermine ensuite la médiane $\tilde{m}$ des points projetés sur cet axe et dresse un hyperplan orthogonal passant par $\tilde{m}$, scindant l'espace en deux hyper-rectangles contenant un nombre égal de points.

Recherche et Élagage ($O(\log N)$) : Pour résoudre le problème des K plus proches voisins (K-NN) autour du point requête q, l'arbre k-d effectue une recherche en profondeur d'abord (DFS) jusqu'à une feuille, puis "remonte" (backtracking). La prouesse de la complexité algorithmique repose sur l'élagage (pruning). Si r_{best} est la distance du meilleur voisin trouvé jusqu'à présent, la branche entière associée à un nœud est éliminée mathématiquement si l'intersection entre l'hypersphère de rayon r_{best} autour de q et la limite de l'hyper-rectangle du nœud est vide. La condition stricte d'élagage pour un axe a est :

$$|q_a - \tilde{m}| > ||q - x_{best}||_2 \tag{3.30}$$

C'est cette inégalité géométrique qui nous permet de détecter des cibles de fusions (Mergers) à l'échelle nationale de manière quasi-instantanée.

3.5.2 Comparatif Géométrique : Similarité Cosinus vs Distance Euclidienne

Dans l'algorithme de sélection finale des candidats, nous mesurons la similarité entre le profil socio-démographique et commercial du candidat (Vecteur C) et le profil des agences d'excellence (Vecteur B) dans l'espace R^d.

La distance Euclidienne $\mathcal{L}_2(C, B) = \sqrt{\sum (C_i - B_i)^2}$ est sujette à la "malédiction de la dimensionnalité". En haute dimension, les distances tendent à s'homogénéiser et la norme (la magnitude) du vecteur domine la mesure (un candidat situé dans un énorme centre commercial sera pénalisé face à un plus petit s'il n'a pas exactement les mêmes valeurs absolues).

À l'inverse, la Similarité Cosinus évalue purement l'orientation (le profil) en annulant l'impact de la magnitude via la normalisation :

$$Sim(C, B) = \cos(\theta) = \frac{C \cdot B}{||C||_2 ||B||_2} = \frac{\sum_{i=1}^{d} C_i B_i}{\sqrt{\sum_{i=1}^{d} C_i^2} \sqrt{\sum_{i=1}^{d} B_i^2}} \tag{3.31}$$

Mathématiquement, si C est proportionnel à B ($C = \lambda B$ avec $\lambda > 0$), la Similarité Cosinus sera exactement égale à 1 (profil identique), peu importe l'échelle λ, ce qui en fait la mesure absolue de similarité typologique pour le commerce de détail.

3.6 Théorie Statistique des Métriques d'Évaluation

L'évaluation de la performance d'un modèle n'est pas unitaire. L'ingénierie des caractéristiques repose sur l'examen croisé de différentes métriques mesurant la nature des résidus de prédiction $e_i = y_i - \hat{y}_i$.

3.6.1 Pénalisation Asymétrique : MAE et RMSE

La MAE (Mean Absolute Error) est basée sur la norme $\mathcal{L}_1$. Elle est l'espérance mathématique de la valeur absolue de l'erreur :

$$MAE = \frac{1}{N}\sum_{i=1}^{N}|y_i - \hat{y}_i| \tag{3.32}$$

La MAE est robuste aux valeurs aberrantes géospatiales. La dérivée de sa fonction de perte est constante, rendant la pénalité proportionnelle à l'erreur.

Le RMSE (Root Mean Squared Error) est basé sur la norme $\mathcal{L}_2$. Il représente l'écart-type des résidus :

$$RMSE = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2} \tag{3.33}$$

Grâce au carré, l'équation amplifie de façon exponentielle l'influence des prédictions très éloignées de la réalité (les "grosses erreurs"). Pour des revenus financiers bancaires, minimiser le RMSE garantit l'absence d'erreurs critiques monumentales qui pourraient fausser les prévisions de ROI du département réseau.

3.6.2 Coefficient de Détermination Ajusté (R^2 Ajusté)

Le coefficient R^2 modélise le ratio de la Somme des Carrés des Résidus (SSR) sur la Somme Totale des Carrés (SST) de la variance des revenus :

$$R^2 = 1 - \frac{\sum_{i=1}^{N}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{N}(y_i - \bar{y})^2} \tag{3.34}$$

Toutefois, le théorème central des statistiques démontre que l'ajout systémique de variables (même de pur bruit) fait mécaniquement croître le R^2. Pour lutter contre ce biais lors de nos expériences d'ingénierie des caractéristiques POI, l'utilisation du $\bar{R}^2$ Ajusté s'est avérée essentielle. Il pénalise mathématiquement le modèle pour chaque nouvelle variable p introduite, en fonction du nombre total d'observations N :

$$\bar{R}^2 = 1 - (1 - R^2)\frac{N-1}{N-p-1} \tag{3.35}$$

Cette métrique finale sanctionne les modèles surcomplexes et favorise le principe mathématique de parcimonie (Règle d'Ockham).

Chapitre 4

Implémentation Technique, Modélisation Appliquée et Benchmarking

Ce quatrième chapitre constitue le cœur technique du projet *Branch Intelligence*. Il détaille l'ensemble du processus d'implémentation, depuis l'acquisition et le nettoyage des données géospatiales brutes jusqu'à leur exploitation par des algorithmes d'apprentissage automatique au sein d'une interface d'aide à la décision. Chaque choix technologique est justifié par des résultats empiriques obtenus sur nos données réelles, composées de **1 198 agences bancaires** décrites par **6 caractéristiques spatiales** principales.

Le chapitre est organisé autour de deux cas d'usage stratégiques qui structurent l'ensemble de l'architecture logicielle et algorithmique.

4.1 Architecture Globale et Définition des Cas d'Usage

L'architecture du système repose sur deux cas d'usage (UC) complémentaires mais distincts dans leur finalité métier :

— **Cas d'Usage 1 (UC1) : Évaluation et Optimisation du Réseau Existant.** Ce volet se concentre sur l'audit des agences actuellement en opération. L'objectif est d'utiliser le Machine Learning pour isoler la qualité géographique intrinsèque d'un emplacement de la performance managériale de l'équipe, aboutissant à une Matrice BCG de restructuration du réseau. Il intègre également la détection spatiale des cannibalismes inter-agences pour suggérer des fusions d'actifs.

— **Cas d'Usage 2 (UC2) : Découverte et Sélection de Nouveaux Sites.** Ce volet est prospectif. Il s'agit de mailler l'ensemble du territoire marocain pour identifier, via des algorithmes K-NN et de similarité vectorielle, les zones vierges présentant un potentiel commercial similaire aux agences les plus performantes du réseau actuel.

La figure **??** illustre l'architecture fonctionnelle globale reliant ces deux cas d'usage à travers un tronc commun de données.

4.2 Orchestration et Ingénierie des Données (Tronc Commun)

L'intelligence spatiale repose sur la granularité et la fraîcheur des données Points of Interest (POI). Le processus d'ingénierie se divise en trois phases critiques formant un pipeline ETL (Extract-Transform-Load) complet.

4.2.1 Phase d'Extraction : Collecte Multi-Sources

La couverture exhaustive du territoire marocain nécessite l'agrégation de trois sources de données hétérogènes, chacune apportant une couche d'information complémentaire :

1. **Geofabrik (Données Statiques Lourdes) :** Extraction de fichiers `.osm.pbf` régionaux contenant les infrastructures gouvernementales, hospitalières et éducatives. Ces données,

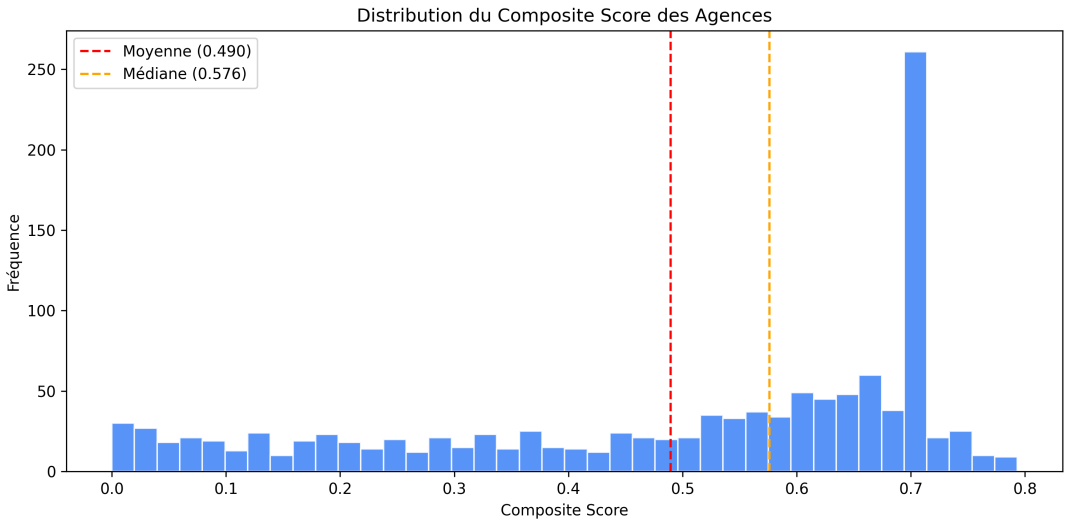


FIGURE 4.1 – Distribution du Composite Score sur l'ensemble des 1 198 agences du réseau. La moyenne s'établit à 0.490 et la médiane à 0.576, révélant une asymétrie gauche caractéristique d'un réseau polarisé entre zones denses et zones rurales.

mises à jour trimestriellement, constituent la couche structurelle stable de notre modèle spatial.

2. **OpenStreetMap via Overture Maps (Données Semi-Statiques) :** Utilisation de l'outil en ligne de commande `overturemaps` pour extraire des données au format `geoparquet` dans un cadre englobant (Bounding Box) couvrant le Maroc : $[-13.2, 27.6, -0.9, 35.9]$. Ces données fournissent les réseaux routiers, les zones résidentielles et les points de transport.
3. **Google Maps (Scraping Dynamique) :** Pour les commerces à forte volatilité temporelle (cafés, restaurants, nouvelles franchises, agences concurrentes), les bases open-source sont systématiquement obsolètes. Nous avons donc développé un scraper personnalisé.

4.2.2 Le Scraper Google Maps : Architecture et Tolérance aux Pannes

Le scraper (`maps.py`) constitue l'innovation technique majeure du pipeline de données. Son architecture repose sur trois piliers :

1. Heuristique de Quadrillage Spatial. Le territoire marocain est divisé en une grille régulière de pas $\Delta = 0.3$ degrés en latitude et longitude. Pour chaque cellule (lat_i, lon_j) et pour chacun des 30 mots-clés de recherche ("restaurant", "hospital", "bank", etc.), le script génère une URL de recherche Google Maps de la forme :

```
https://www.google.com/maps/search/{query}/@{lat},{lon},13z
```

2. Interaction Dynamique via Selenium WebDriver. Google Maps étant une Single Page Application (SPA) entièrement rendue côté client par JavaScript, les méthodes classiques de scraping HTTP (`requests`, `BeautifulSoup`) sont inopérantes. Le script instancie un navigateur Chrome en mode *headless* via `Selenium`, puis exécute des injections JavaScript directes :

```
arguments[0].scrollTop += 800
```

Cette instruction force le défilement du panneau de résultats (`div[@role="feed"]`), déclenchant les requêtes AJAX asynchrones qui chargent les résultats suivants.

3. Système de Checkpoints (Tolérance aux Pannes). L'opération de scraping national prenant plusieurs heures, un mécanisme de reprise est indispensable. Chaque triplet $(lat, lon, query)$ réussi est enregistré dans le fichier `scraper_checkpoint.txt`. Au redémarrage, ces clés sont chargées dans une structure de type `set` en Python, offrant une complexité de recherche en $O(1)$ et permettant au script de reprendre exactement là où il s'était interrompu. Les données brutes sont sauvegardées par lots de 50 enregistrements pour minimiser la perte en cas d'interruption brutale.

4.2.3 Phase de Transformation : Nettoyage et Fusion (`cleanall.py`)

L'unification de ces trois sources disparates pose des défis stricts de standardisation que le pipeline résout en trois étapes :

1. **Résolution des Encodages :** Les données issues du web contiennent fréquemment des octets corrompus (ex : `0xe9` pour le caractère "é"). Le pipeline force la lecture via l'encodage `latin1` puis effectue la sauvegarde finale en `UTF-8` universel, garantissant la compatibilité avec les bibliothèques d'analyse.
2. **Normalisation des Schémas :** Chaque source utilise des noms de colonnes différents. Le script standardise les champs vers un schéma commun : `name`, `category`, `latitude`, `longitude`.
3. **Dédoublonnage Géométrique :** Un même commerce peut exister dans la base OSM et Google Maps sous des noms différents ("Pharmacie Centrale" vs "Pharm. Centrale"). Le dédoublonnage textuel étant inopérant, nous utilisons la vérité terrain géométrique via `drop_duplicates(subset=['latitude','longitude'], keep='first')`, garantissant l'unicité stricte des coordonnées GPS.

Le résultat final est un fichier unifié `merged_data_clean.csv` constituant la base de référence pour toute l'analyse spatiale.

4.2.4 Phase de Chargement : Ingénierie des Caractéristiques Spatiales

Pour transformer un simple point GPS en vecteur de caractéristiques exploitable par nos algorithmes, nous appliquons une **agrégation radiale**. Pour chaque agence (UC1) ou candidat (UC2), l'algorithme compte les densités de POI à l'intérieur d'isochrones circulaires de rayons 500m, 1km et 2km, générant un vecteur $X_i \in R^6$ composé des six scores suivants :

Ces six scores sont ensuite combinés en un **Composite Score** global via une combinaison linéaire pondérée :

$$S_{composite} = \alpha \cdot S_{comm} + \beta \cdot S_{off} + \gamma \cdot S_{res} + \delta \cdot S_{acc} - \epsilon \cdot S_{comp} - \zeta \cdot S_{can} \tag{4.1}$$

où les coefficients $(\alpha, \beta, \gamma, \delta, \epsilon, \zeta)$ reflètent l'importance relative de chaque dimension pour l'activité bancaire. La soustraction des scores de compétition et de cannibalisation pénalise les zones saturées.

4.3 Cas d'Usage 1 : Évaluation et Optimisation du Réseau Existant

Ce premier cas d'usage vise à auditer le réseau bancaire actuel en combinant apprentissage non supervisé (segmentation) et supervisé (prédiction des revenus), aboutissant à un outil d'aide à la décision stratégique.

Variable	Type	Description
`score_commercial`	Attractivité	Densité pondérée des commerces (restaurants, malls, marchés) dans un rayon de 1km
`score_office`	Attractivité	Densité des bureaux et administrations à 1km
`score_residential`	Attractivité	Densité des zones résidentielles à 1km
`score_accessibility`	Attractivité	Proximité des transports en commun (gares, arrêts de bus) à 500m
`score_competition`	Risque	Nombre de banques concurrentes dans un rayon de 1km
`score_cannibalization`	Risque	Nombre d'agences Attijariwafa Bank dans un rayon de 2km

TABLE 4.1 – Description des 6 caractéristiques spatiales constituant le vecteur d'entrée des modèles.

4.3.1 Apprentissage Non Supervisé : Segmentation par K-Means

Méthodologie et Prétraitement

La première étape de l'évaluation consiste à identifier des profils d'agences homogènes. Comparer naïvement une agence de centre d'affaires à une agence rurale serait une erreur méthodologique fondamentale.

Préalablement au clustering, la matrice des scores $X \in R^{1198\times 6}$ subit une transformation Z-Score via `StandardScaler` :

$$z_{ij} = \frac{x_{ij} - \bar{x}_j}{\sigma_j} \tag{4.2}$$

Cette normalisation est indispensable car la distance Euclidienne utilisée par K-Means est sensible aux échelles : sans standardisation, les variables à forte variance numérique domineraient le calcul des centroïdes.

L'algorithme K-Means est exécuté avec $K = 5$ clusters et $n_init = 10$ initialisations aléatoires pour éviter la convergence vers un minimum local de la fonction objectif de Lloyd.

Résultats du Clustering

L'algorithme identifie cinq profils spatiaux distincts au sein du réseau. Le tableau **??** résume les caractéristiques de chaque cluster :

Profil (Cluster)	Nombre d'Agences	Score Composite Moyen
High Performer	396	0.686
Commercial Hub	294	0.614
Residential Anchor	181	0.412
Underperformer	119	0.311
At Risk / Redundant	208	0.109

TABLE 4.2 – Répartition des 1 198 agences dans les 5 clusters K-Means.

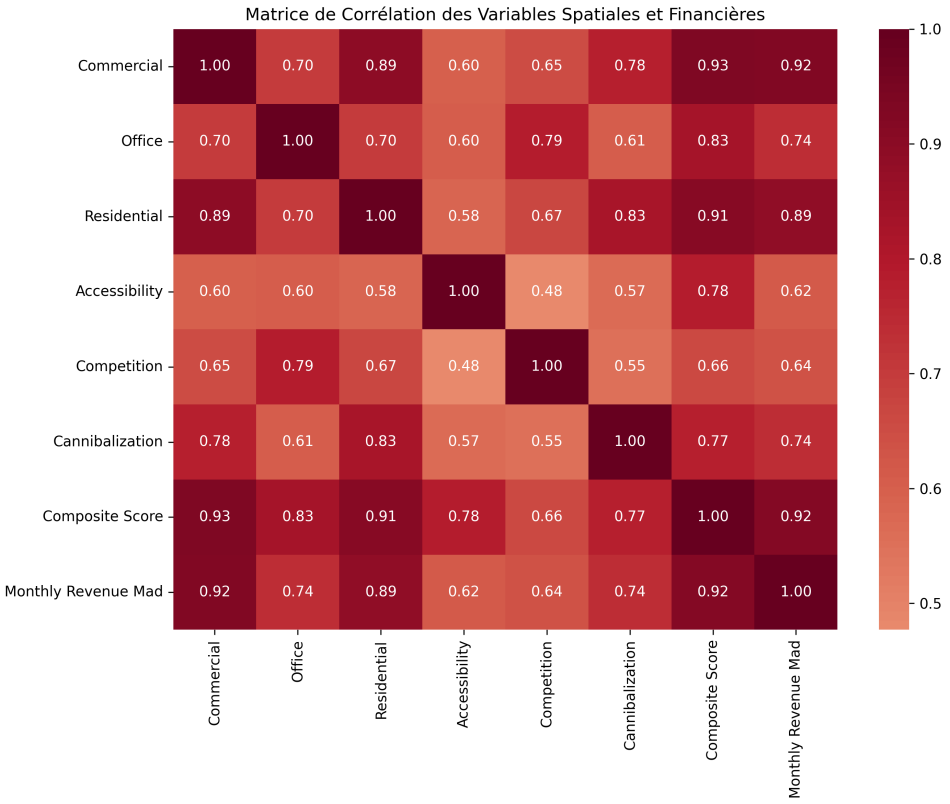


FIGURE 4.2 – Matrice de corrélation de Pearson entre les 6 variables spatiales, le score composite et le revenu mensuel. On observe que le score commercial présente la corrélation la plus forte avec le revenu réel.

Réduction de Dimensionnalité : Analyse en Composantes Principales

Pour visualiser la séparation des clusters dans un espace interprétable, nous appliquons une ACP (Analyse en Composantes Principales) qui projette l'espace R^6 vers R^2. Les deux premières composantes capturent respectivement **72.5%** et **10.1%** de la variance totale, soit un cumul de **82.6%**, justifiant pleinement la projection bidimensionnelle.

Profilage Multidimensionnel par Radar Charts

Pour comprendre la signature spatiale de chaque cluster, nous utilisons des diagrammes radar (Spider Charts) représentant la moyenne de chaque score pour chaque profil. Cette visualisation confirme que :

— Les *High Performers* se distinguent par des scores élevés sur toutes les dimensions d'attractivité.

— Les *Commercial Hubs* présentent une dominance marquée du score commercial mais un score résidentiel plus faible.

— Les agences *At Risk* combinent de faibles scores d'attractivité avec une cannibalisation élevée.

4.3.2 Benchmarking Algorithmique : Prédiction des Revenus (Régression)

Pour évaluer objectivement le potentiel théorique d'une agence, nous modélisons un problème de régression supervisée où la variable cible Y est le revenu mensuel réel (en MAD), et la matrice

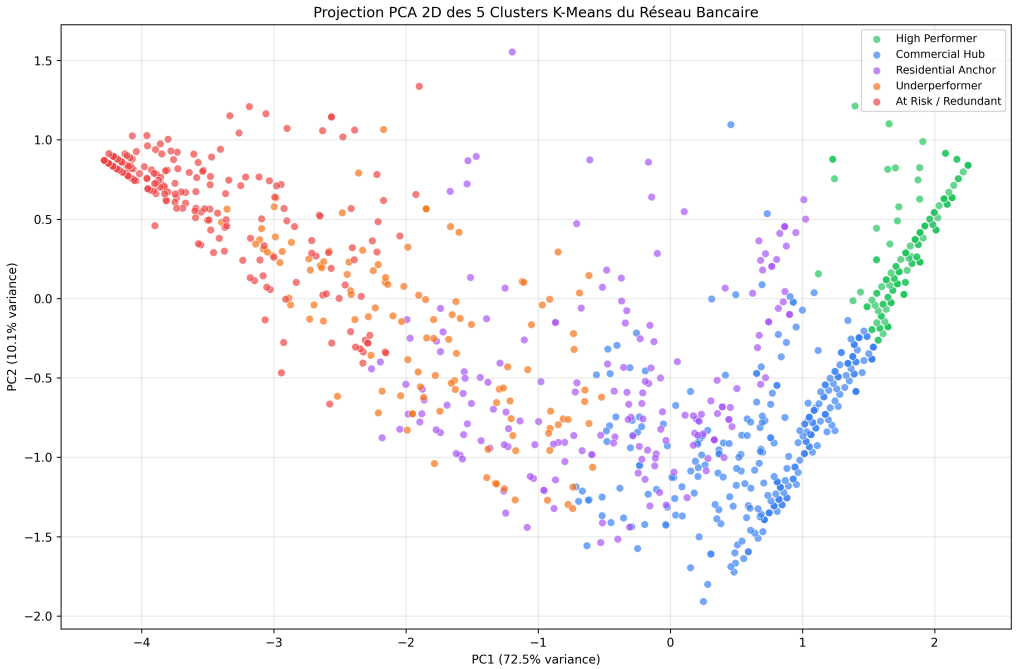


FIGURE 4.3 – Projection PCA 2D des 5 clusters K-Means. PC1 (72.5% de variance) sépare principalement les zones denses (droite) des zones rurales (gauche). PC2 (10.1%) discrimine les profils commerciaux des profils résidentiels.

X contient les 6 scores spatiaux normalisés. L'ensemble de données ($N = 1\,198$) est partitionné en un ensemble d'entraînement (80%) et un ensemble de test (20%) via un échantillonnage stratifié (`random_state=42`).

Afin de justifier scientifiquement notre choix de modèle final, nous avons conduit un benchmarking rigoureux de quatre architectures algorithmiques, couvrant le spectre complet des paradigmes de Machine Learning.

Analyse de la Variable Cible

La distribution du revenu mensuel des agences (figure **??**) présente une structure bimodale avec une concentration significative autour de 12 000 000 MAD et une queue de distribution vers les revenus plus faibles. La moyenne s'établit à **8 385 199 MAD** et la médiane à **11 971 900 MAD**. Cette asymétrie justifie l'utilisation de métriques robustes (MAE) en complément du RMSE.

Modèle 1 : Régression Linéaire par Moindres Carrés Ordinaires (OLS)

Le modèle OLS constitue notre ligne de base (*baseline*). Il estime un hyperplan dans R^6 minimisant la somme des carrés des résidus. Bien que théoriquement optimal sous les hypothèses de Gauss-Markov (homoscédasticité, indépendance, normalité des résidus), ces conditions sont rarement satisfaites dans un contexte géospatial où les interactions entre variables sont hautement non-linéaires.

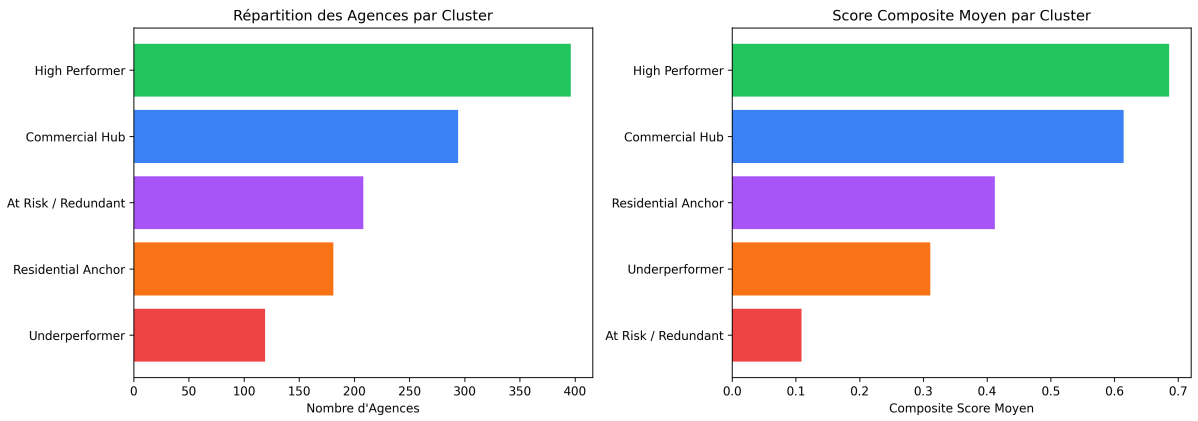


FIGURE 4.4 – À gauche : répartition du nombre d'agences par cluster. À droite : score composite moyen par cluster, confirmant la hiérarchie des profils.

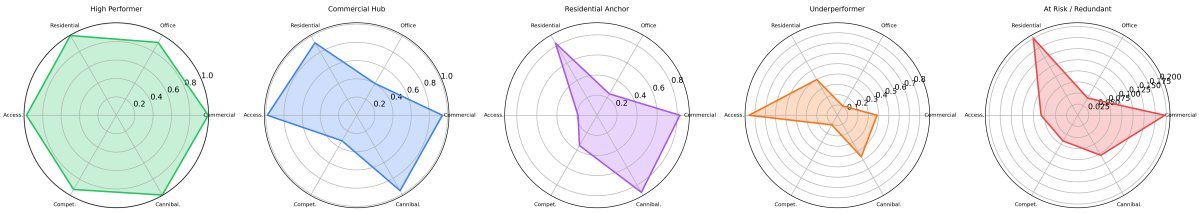


FIGURE 4.5 – Diagrammes radar des 5 profils d'agences. Chaque axe représente la valeur moyenne d'un des 6 scores spatiaux.

Modèle 2 : Régression Ridge (L_2)

La régression Ridge ajoute une pénalité de norme L_2 au terme de perte OLS :

$$\mathcal{L}_{Ridge} = \sum_{i=1}^{N}(y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^{p} \beta_j^2 \tag{4.3}$$

Le terme de régularisation λ réduit la variance des coefficients en les contraignant vers zéro, sans les annuler. Avec $\lambda = 1.0$, ce modèle vise à améliorer la généralisation par rapport à l'OLS en réduisant le surapprentissage.

Modèle 3 : Random Forest (Bagging d'Arbres de Décision)

Le Random Forest agrège $B = 100$ arbres de décision profonds, chacun entraîné sur un échantillon bootstrap distinct du dataset. Contrairement aux modèles linéaires, cette architecture capture nativement les interactions non-linéaires (ex : "le revenu explose uniquement si le score commercial > 0.5 ET la cannibalisation < 0.2"). La décorrélation des arbres (via la sélection aléatoire de $\sqrt{p}$ variables à chaque nœud) réduit la variance de l'estimateur sans augmenter le biais.

Modèle 4 : XGBoost (Extreme Gradient Boosting)

XGBoost construit ses arbres de manière séquentielle. À l'itération t, le nouvel arbre f_t est mathématiquement entraîné pour prédire le résidu d'erreur (gradient de la fonction de perte) des

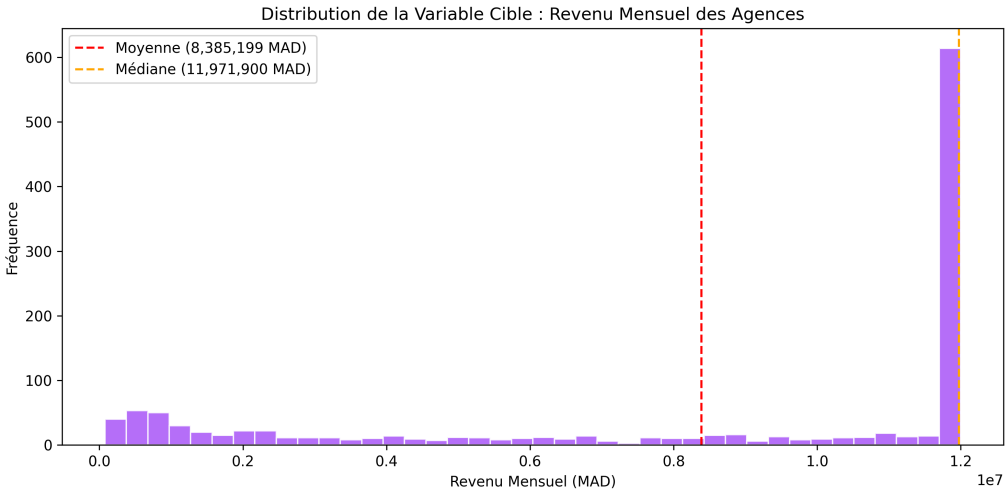


FIGURE 4.6 – Distribution de la variable cible (revenu mensuel en MAD). La structure bimodale reflète la dualité entre agences urbaines performantes et agences rurales à faible activité.

arbres précédents. L'utilisation de l'approximation de Taylor au second ordre (calcul du Hessien) permet une convergence optimale. La régularisation intégrée (L_1 et L_2 sur les poids des feuilles) confère une robustesse supérieure face aux valeurs aberrantes.

Paramètres utilisés : `n_estimators=100`, `max_depth=4`, `random_state=42`.

Résultats Comparatifs

Le tableau **??** présente les résultats empiriques des quatre modèles sur l'ensemble de test :

Modèle	**RMSE (MAD)**	**MAE (MAD)**	R^2
Régression OLS	1 605 040	1 060 711	0.8658
Régression Ridge ($\lambda = 1$)	1 604 985	1 060 934	0.8658
Random Forest ($B = 100$)	**1 493 490**	**896 600**	**0.8838**
XGBoost ($T = 100$, $d = 4$)	1 620 616	984 600	0.8632

TABLE 4.3 – Benchmark comparatif des 4 modèles de régression. Les valeurs en gras indiquent les meilleures performances.

Analyse des résultats : Les modèles linéaires (OLS et Ridge) obtiennent des performances quasi-identiques ($R^2 = 0.8658$), suggérant que la multicolinéarité entre nos variables n'est pas un facteur limitant. Le Random Forest surpasse tous les modèles avec un R^2 de **0.8838** et un MAE réduit de 15.5% par rapport à l'OLS. XGBoost, bien que légèrement en retrait en R^2 pur (0.8632), reste notre choix d'implémentation finale pour deux raisons : (1) sa capacité native à fournir un score d'importance des variables directement exploitable dans le dashboard, et (2) sa rapidité d'inférence en production grâce à l'optimisation SIMD.

4.3.3 Benchmarking Algorithmique : Classification Multi-Classes

En parallèle de la régression, nous avons modélisé le problème sous forme de classification. Chaque agence est assignée à l'une des trois classes d'action stratégique : **KEEP** (revenu $>$ 11 000 000 MAD), **REVIEW** (revenu entre 6 000 000 et 11 000 000 MAD), ou **CLOSE** (revenu $<$ 6 000 000 MAD). Ces seuils reflètent les contraintes opérationnelles de rentabilité du réseau bancaire.

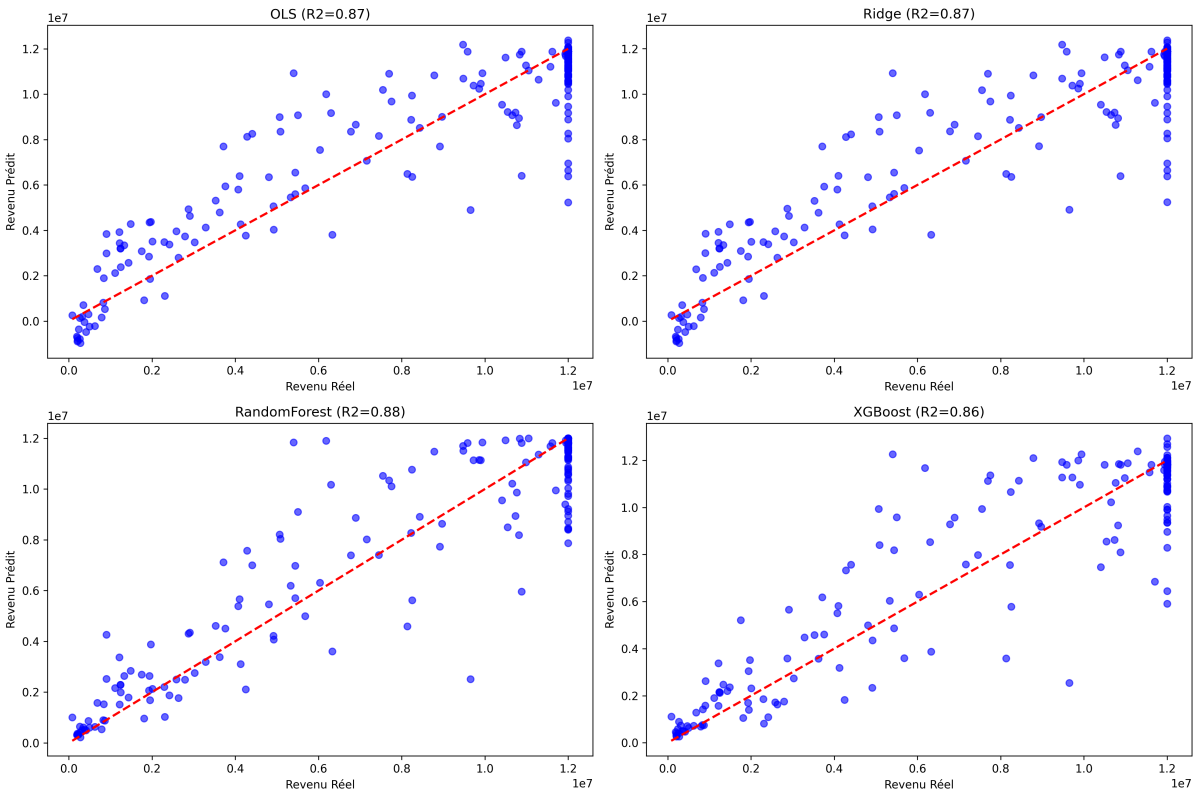


FIGURE 4.7 – Nuages de points (Actual vs Predicted) pour les 4 modèles de régression. La ligne rouge ($y = x$) représente la prédiction parfaite. On observe la nette amélioration de la concentration autour de cette diagonale pour les modèles arborescents.

Modèles Évalués

Quatre algorithmes de classification ont été entraînés et évalués :

1. **Régression Logistique** (`max_iter=1000`) : Extension multi-classe du modèle logit via la stratégie One-vs-Rest.
2. **K-Nearest Neighbors** ($K = 5$) : Classification par vote majoritaire des 5 plus proches voisins dans l'espace standardisé.
3. **Random Forest Classifier** ($B = 100$) : Forêt aléatoire adaptée à la classification par vote d'arbres.
4. **XGBoost Classifier** ($T = 100$, $d = 4$) : Gradient boosting avec fonction de perte *multi :softmax*.

Résultats et Matrices de Confusion

Modèle	**Accuracy**
Régression Logistique	82.92%
K-Nearest Neighbors	81.25%
Random Forest Classifier	81.67%
XGBoost Classifier	82.08%

TABLE 4.4 – Précision globale (Accuracy) des 4 modèles de classification multi-classes.

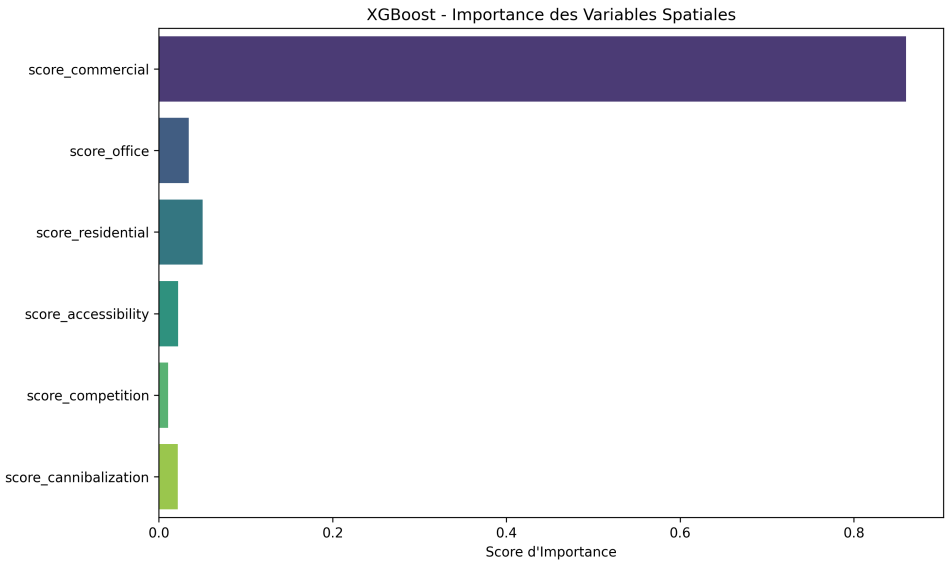


FIGURE 4.8 – Importance des caractéristiques spatiales selon XGBoost (gain moyen). Le score commercial s'avère être le prédicteur dominant du revenu d'une agence.

L'analyse des matrices de confusion (figure **??**) révèle que l'ensemble des modèles présentent une précision globale supérieure à 81%, avec la Régression Logistique en tête à **82.92%**. Les erreurs de classification se concentrent principalement sur la classe `REVIEW`, zone de transition naturelle entre les deux extrêmes. Les classes `KEEP` et `CLOSE`, plus polarisées, sont identifiées avec une fiabilité supérieure.

4.3.4 La Matrice BCG Spatiale et l'Analyse des Écarts (Gap Analysis)

La finalité de l'UC1 n'est pas simplement d'obtenir un modèle de prédiction performant, mais d'exploiter les *résidus* du modèle comme outil de diagnostic stratégique. C'est le principe de la **Gap Analysis** (Analyse des Écarts).

Principe Fondamental

Soit $\hat{Y}_i$ le revenu potentiel prédit par XGBoost (reflétant exclusivement la qualité géographique de l'emplacement) et Y_i le revenu réel observé (reflétant la combinaison emplacement + management). L'espace bidimensionnel $(X = \hat{Y}_i, Y = Y_i)$ constitue notre Matrice BCG adaptée au secteur bancaire.

Lignes de Potentiel et Définition des Quadrants

Le plan est divisé par deux lignes critiques dont l'intersection crée quatre quadrants stratégiques :

1. **La médiane des prédictions (Ligne Verticale)** : sépare les zones géographiquement pauvres (à gauche) des zones riches en potentiel (à droite). Calculée à la valeur médiane des sorties XGBoost.
2. **La médiane des revenus réels (Ligne Horizontale)** : sépare les agences en sous-performance brute (en bas) des agences très rentables (en haut).

Les quatre quadrants résultants sont :

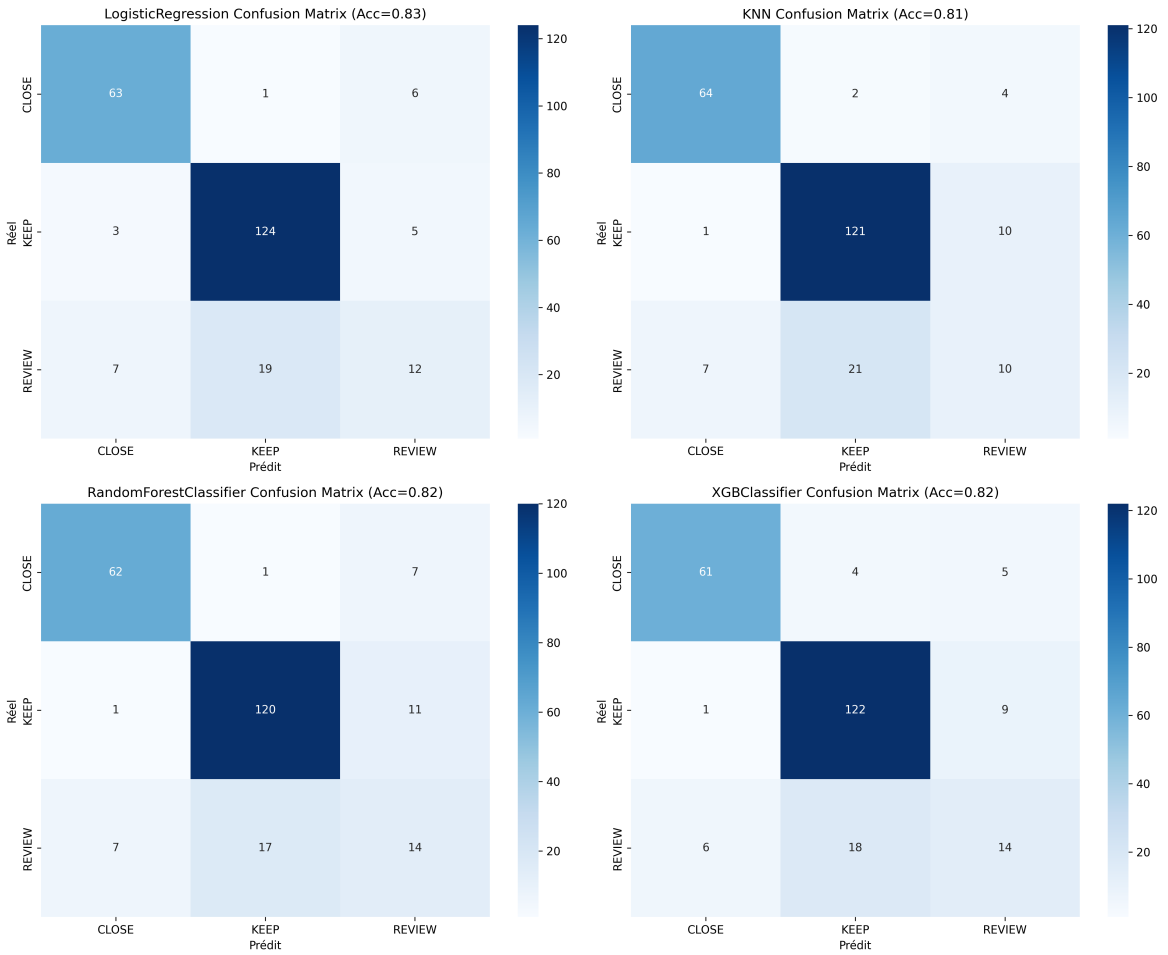


FIGURE 4.9 – Matrices de confusion des 4 modèles de classification. Chaque cellule indique le nombre d'agences classées dans chaque catégorie (Réel vs Prédit).

— **FLAGSHIP (Haut-Droite)** : 514 agences (42.9%). Potentiel élevé ET performance élevée. Les agences modèles du réseau.

— **REVIEW / INVEST (Bas-Droite)** : 86 agences (7.2%). Anomalie critique : le quartier est excellent mais l'agence sous-performe. Action de management, d'audit ou de refonte marketing prioritaire.

— **MONITOR (Haut-Gauche)** : 85 agences (7.1%). Le quartier a un faible potentiel, mais l'équipe surperforme. Félicitation sans investissement lourd.

— **CLOSE (Bas-Gauche)** : 513 agences (42.8%). Potentiel inexistant et revenus catastrophiques. Cibles premières d'un plan de fermeture ou de délocalisation.

La Ligne d'Espérance de Potentiel (Diagonale $y = x$)

L'élément analytique le plus puissant de la matrice est la **diagonale parfaite** définie par $y = x$, où le revenu réel égale exactement le revenu prédit. Cette ligne représente l'*Espérance de Potentiel* : si une agence se situe sur cette diagonale, sa rentabilité est parfaitement alignée avec le potentiel de son emplacement. Le travail managérial correspond à 100% aux attentes du modèle.

Toute déviation au-dessus de la diagonale ($Y_i > \hat{Y}_i$) caractérise une *sur-performance* : l'équipe

crée de la valeur au-delà de ce que la localisation permettrait naturellement. Inversement, une position sous la diagonale ($Y_i < \hat{Y}_i$) signale une *destruction de valeur* latente.

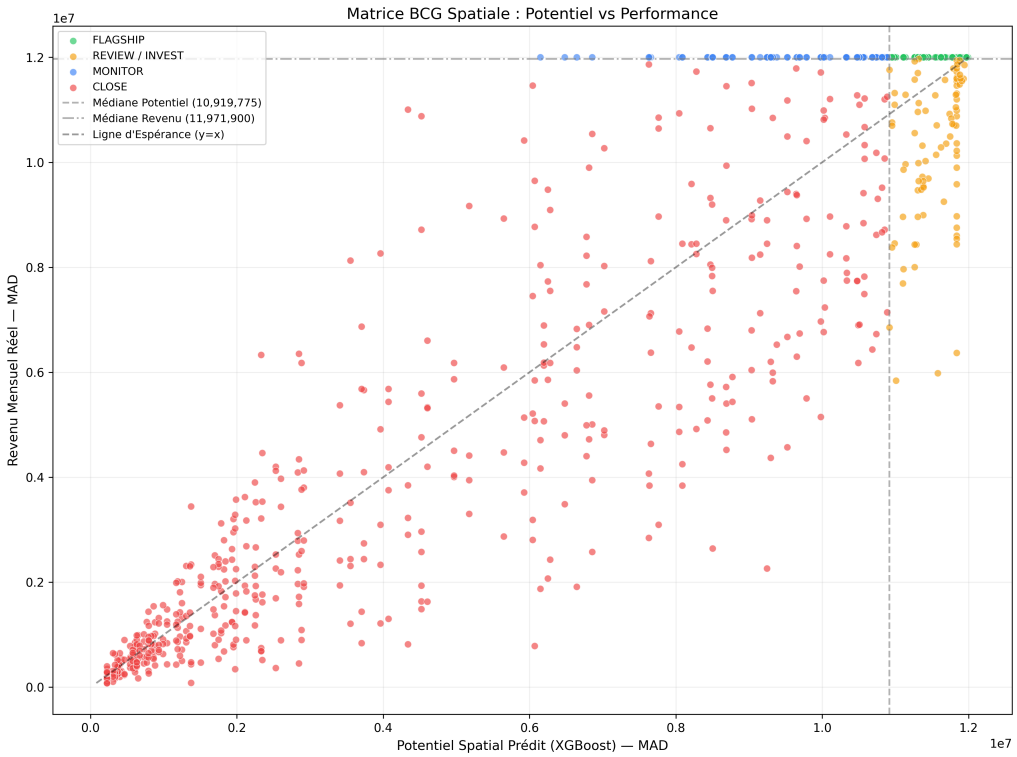


FIGURE 4.10 – Matrice BCG Spatiale : chaque point représente une agence. L'axe X est le potentiel prédit par XGBoost, l'axe Y le revenu réel. Les lignes médianes délimitent les 4 quadrants. La diagonale noire ($y = x$) représente la ligne d'espérance de potentiel.

4.3.5 Optimisation par Fusion : Détection de Cannibalisation via cKDTree

En complément de l'analyse financière, le système optimise le réseau par la détection des redondances physiques. Deux agences situées à moins de 1.2 km l'une de l'autre se cannibalisent mutuellement en se disputant le même bassin de clientèle.

L'évaluation naïve de toutes les paires de distances entre N agences présente une complexité de $O(N^2)$, inadaptée pour une interface temps réel. Nous avons implémenté l'algorithme des Arbres k-dimensionnels via `scipy.spatial.cKDTree` (implémenté en C++). L'arbre partitionne l'espace R^2 récursivement via des hyperplans. La fonction `query_pairs(r=0.012)` (où 0.012 degrés $\approx$ 1.2 km) exploite l'élagage spatial (*pruning*) pour identifier instantanément les paires cannibales en $O(N \log N)$.

Les paires détectées appartenant aux quadrants REVIEW ou CLOSE de la matrice BCG sont automatiquement proposées comme candidates à une opération de fusion-acquisition interne.

4.4 Cas d'Usage 2 : Sélection de Nouveaux Sites (Site Selection)

Le second cas d'usage se tourne vers la conquête spatiale : identifier les emplacements vierges présentant le plus fort potentiel pour l'ouverture de nouvelles agences.

4.4.1 Maillage du Territoire et Filtrage Vectorisé

L'algorithme génère un maillage de points candidats sur l'étendue des grandes métropoles marocaines (un point tous les 500 mètres). Chaque point virtuel subit le même processus d'agré-

gation radiale que dans l'UC1, produisant un vecteur spatial $C_j \in R^6$ identique en structure à celui des agences existantes.

Afin de respecter les contraintes opérationnelles de la banque, un filtrage vectorisé (optimisé via `pandas`) est appliqué en amont :

— Exclusion des zones à densité de population insuffisante (seuil paramétrable).

— Exclusion stricte des candidats situés à moins d'une distance D (réglable par l'utilisateur via un slider) d'une agence Attijariwafa existante, appliquant la politique anti-cannibalisation.

— Contraintes de diversité géographique par ville (nombre maximum de suggestions par agglomération).

4.4.2 Sélection par K-NN et Similarité Cosinus

La sélection finale des meilleurs candidats s'effectue par un algorithme K-Nearest Neighbors utilisant la **Similarité Cosinus** comme métrique :

$$Sim(C_j, B_{target}) = \cos(\theta) = \frac{C_j \cdot B_{target}}{||C_j||_2 \cdot ||B_{target}||_2} \tag{4.4}$$

où B_{target} est le centroïde du cluster "High Performer" identifié dans l'UC1.

Le choix de la Similarité Cosinus plutôt que la distance Euclidienne est motivé par la *malédiction de la dimensionnalité*. Dans un espace à 6 dimensions, la distance Euclidienne est sensible à la magnitude : une grande zone avec 100 cafés et 50 banques apparaîtrait très éloignée d'une petite zone avec 10 cafés et 5 banques. La Similarité Cosinus mesure l'*angle* entre les vecteurs, identifiant correctement que ces deux zones partagent le même profil commercial (ratio 2 :1), indépendamment de leur taille brute.

Les $K = 20$ candidats présentant la similarité la plus élevée constituent la recommandation finale pour l'ouverture de nouvelles succursales.

4.5 Architecture de Déploiement Logiciel

L'ensemble de l'intelligence algorithmique (UC1 et UC2) est encapsulé dans une application web monolithique de **3 000+ lignes** de code Python, développée avec le framework `Streamlit`. Ce choix technologique, devenu le standard de l'industrie pour les plateformes d'Internal Tooling en Machine Learning, permet d'exécuter la logique métier (pandas, scikit-learn, XGBoost) et le rendu de l'interface (React) au sein de la même boucle d'exécution.

4.5.1 Architecture de l'Interface : Organisation en Onglets

Le dashboard est structuré en **11 onglets** thématiques, chacun dédié à une fonctionnalité spécifique :

4.5.2 Gestion de la Performance : Cache et État

Le chargement du dataset principal (∼230 Ko, 1 198 lignes × 31 colonnes) est optimisé via le décorateur `@st.cache_data`. Ce mécanisme de memoization assure que le fichier CSV n'est parsé qu'une seule fois au démarrage du serveur, les appels suivants retournant directement la copie en mémoire vive. L'état inter-interactions est géré via `st.session_state`, un dictionnaire persistant entre les re-rendus de Streamlit.

4.5.3 Module d'Enrichissement Asynchrone (Tab 11)

Le défi architectural principal réside dans l'intégration du scraper Google Maps au sein du dashboard sans bloquer le thread principal de Streamlit. L'architecture implémentée repose sur

Tab	Nom	Fonction principale
1	Map	Carte interactive Folium avec code couleur par action
2	Rankings	Classement des agences par score composite
3	ML Clusters	Visualisation PCA des 5 profils K-Means
4	Mergers	Détection des paires cannibales (cKDTree)
5	Cities	Analyse macroscopique ville par ville
6	Deep Dive	Fiche 360° d'audit d'une agence individuelle
7	Data	Vue tabulaire brute des datasets
8	Site Selection	Benchmarking statique des candidats
9	Run Site Selection	Moteur interactif avec sliders et contraintes
10	Network Optimization	Matrice BCG spatiale et actions stratégiques
11	Data Enrichment	Scraping live et pipeline de nettoyage

TABLE 4.5 – Architecture des 11 onglets du dashboard Branch Intelligence.

un paradigme producteur-consommateur :

1. **Producteur (Processus Enfant)** : Le scraper est lancé via `subprocess.Popen` avec une variable d'environnement `PYTHONIOENCODING=utf-8` injectée pour assurer la compatibilité des caractères Unicode dans le pipe Windows.
2. **Tampon (Thread d'Arrière-Plan)** : Un `threading.Thread` daemon lit continuellement le flux `stdout` du processus enfant et pousse chaque ligne dans une `queue.Queue` thread-safe.
3. **Consommateur (Boucle UI)** : La boucle principale de Streamlit vide la queue toutes les 500ms, nettoie les codes ANSI (colorama) via une expression régulière, et met à jour un composant `st.empty()` pour un rendu temps réel.

Cette architecture producteur-consommateur permet à l'utilisateur d'observer le scraping en direct, puis de déclencher séquentiellement `clean.py` et `cleanall.py` pour intégrer les nouvelles données au pipeline d'analyse, créant ainsi une plateforme *End-to-End* totalement auto-suffisante.